

Mohamad SAMMAN

September 5, 2025

Contents

1	Introduction	4
2	Optimal Control and the Pontryagin Maximum Principle	4
2.1	Introduction to Optimal Control	4
2.2	Mathematical Formulation of Optimal Control Problems	5
2.3	The Pontryagin Maximum Principle (PMP)	5
2.4	Conclusion	6
3	Description of the System	6
3.1	Parameterized and Observed Nonlinear Control Problem	7
3.2	Parameter Sensitivity	7
3.3	Quantifying the Deviation Between y and y_{meas}	7
4	The Methods	9
4.1	Method 1: energy constraint	9
4.2	Method 2: energy constraint with Newton-Raphson method	11
4.3	The optimization algorithm	13
5	Numerical Results and Analysis of the preexisting methods	14
6	Experiment design	16
6.1	Defining the problem	16
6.2	Computing the information matrix	17
7	Explanation of the Python Script (tools.py)	18
7.1	Function Overview	19
7.2	Functions for Solving Differential Equations	19
7.3	Matrix Manipulation Functions	19
7.4	Auxiliary geometric functions	20
7.5	Optimal control computation	21
7.6	Additional functions	21
8	Results	22
9	Conclusion	24

10 Annexes	26
10.1 Codes sources	26

Acknowledgements:

I would like to thank all the people who supported and guided me during my internship. I would also like to thank my teachers at University Côte d'Azur for their help and guidance. Special thanks to my family for supporting me during my studies and internship.

1 Introduction

In this report I will present the work I did during my internship at INRIA Sophia-Antipolis. **INRIA** (National Research Institute for Digital Science and Technology) is a French public research institution that focuses on computer science, mathematics, and digital technologies. The Inria Sophia Antipolis-Méditerranée center, located in the Sophia Antipolis region, is one of its key research hubs. This center, in collaboration with **Université Côte d’Azur**, is renowned for its expertise in areas such as artificial intelligence, algorithms, cybersecurity, and embedded systems. Inria also maintains strong partnerships with national and international institutions. My internship combined optimal control and inverse analysis for the design of a neuroscience experiment. I started by studying state of the art optimal control for linear system identification and then implemented a python script for finding an optimal control for experiment design.

2 Optimal Control and the Pontryagin Maximum Principle

Optimal control lies at the intersection of dynamical systems and optimization, seeking control inputs that drive a system from an initial condition to a desired final state in a manner that optimizes a given cost criterion. This framework naturally arises in numerous scientific and engineering contexts where systems evolve over time under constrained decision variables.

One of the central theoretical tools in optimal control is the Pontryagin Maximum Principle (PMP), which provides necessary conditions that any optimal control and corresponding state trajectory must satisfy. Unlike classical optimization, where solutions can often be found by setting gradients to zero, optimal control problems involve infinite-dimensional spaces of functions. The PMP reformulates such problems into a set of coupled ordinary differential equations with boundary conditions, paving the way for both analytical and numerical solution methods.

In what follows, we introduce the basic setup of optimal control problems, outline their mathematical formulation, and present the Pontryagin Maximum Principle in detail. Through this development, we highlight how the PMP bridges the gap between control theory and variational calculus, and how it serves as a foundational tool for solving real-world dynamic optimization problems.

2.1 Introduction to Optimal Control

Optimal control theory provides a rigorous mathematical framework for determining control policies that steer dynamical systems over time while optimizing a prescribed performance criterion. It extends the classical calculus of variations by explicitly incorporating control variables that directly influence the state evolution via ordinary differential equations (ODEs), and it offers a natural setting for dealing with constraints and discontinuities in control.

Let a system be governed by a set of nonlinear ODEs:

$$\dot{x}(t) = f(x(t), u(t), t), \quad x(t_0) = x_0,$$

where $x(t) \in \mathbb{R}^n$ denotes the state vector, $u(t) \in \mathbb{R}^m$ is the control input, and $f : \mathbb{R}^n \times \mathbb{R}^m \times [t_0, t_f] \rightarrow \mathbb{R}^n$ is a smooth vector field describing the system dynamics.

The objective is to determine an admissible control $u(t)$ such that the state trajectory $x(t)$ minimizes a cost functional of the form:

$$J[u(\cdot)] = \int_{t_0}^{t_f} L(x(t), u(t), t) dt + \phi(x(t_f)),$$

where $L : \mathbb{R}^n \times \mathbb{R}^m \times [t_0, t_f] \rightarrow \mathbb{R}$ is the running cost and $\phi : \mathbb{R}^n \rightarrow \mathbb{R}$ is the terminal cost.

This optimal control formulation is applicable to a wide range of real-world problems, including spacecraft trajectory optimization, financial asset allocation, pharmacokinetics, and autonomous navigation.

2.2 Mathematical Formulation of Optimal Control Problems

Formally, an optimal control problem (OCP) may be posed as:

$$\begin{aligned} \text{Minimize: } & J[u(\cdot)] = \int_{t_0}^{t_f} L(x(t), u(t), t) dt + \phi(x(t_f)) \\ \text{subject to: } & \dot{x}(t) = f(x(t), u(t), t), \quad x(t_0) = x_0, \\ & x(t) \in \mathcal{X} \subseteq \mathbb{R}^n, \quad u(t) \in \mathcal{U} \subseteq \mathbb{R}^m, \quad t \in [t_0, t_f]. \end{aligned}$$

Here, $\mathcal{X}$ and $\mathcal{U}$ denote the sets of admissible states and controls, respectively, which may encode physical limitations such as actuator bounds or safety margins.

The goal is to synthesize a control policy $u^*(t) \in \mathcal{U}$ that produces an admissible state trajectory $x^*(t) \in \mathcal{X}$ and minimizes the objective functional J . Solutions may involve continuous, discontinuous, or even impulsive control strategies depending on the nature of the system and constraints.

2.3 The Pontryagin Maximum Principle (PMP)

The Pontryagin Maximum Principle, developed in the mid-20th century, provides first-order necessary conditions for optimality in deterministic control systems. It transforms the infinite-dimensional optimization problem into a boundary-value problem (BVP) for a system of differential equations.

2.3.1 The Hamiltonian Function

To derive the PMP, we introduce the *Hamiltonian* function, which plays a central role in formulating the necessary conditions for optimality:

$$H(x, u, \lambda, t) = \lambda^\top f(x, u, t) + L(x, u, t),$$

where $\lambda(t) \in \mathbb{R}^n$ is the *costate* or *adjoint variable*, representing the sensitivity of the optimal cost with respect to the state variables.

2.3.2 Statement of the Pontryagin Maximum Principle

Let $u^*(t)$ be an optimal control, and $x^*(t)$ the associated optimal trajectory. Then there exists a nontrivial absolutely continuous costate trajectory $\lambda(t)$ such that, for almost every $t \in [t_0, t_f]$, the following conditions hold:

1. **State dynamics:**

$$\dot{x}^*(t) = \frac{\partial H}{\partial \lambda}(x^*(t), u^*(t), \lambda(t), t) = f(x^*(t), u^*(t), t),$$

2. **Costate dynamics:**

$$\dot{\lambda}(t) = -\frac{\partial H}{\partial x}(x^*(t), u^*(t), \lambda(t), t),$$

3. **Maximum condition:**

$$u^*(t) \in \arg \max_{u \in \mathcal{U}} H(x^*(t), u, \lambda(t), t),$$

4. **Transversality conditions:** These depend on whether the final time or final state is fixed, free, or constrained. For instance, if the final state $x(t_f)$ is free, then $\lambda(t_f) = \nabla \phi(x^*(t_f))$.

These necessary conditions characterize the structure of optimal trajectories and can be interpreted geometrically as an extension of classical mechanics into the optimal control domain.

2.4 Conclusion

The Pontryagin Maximum Principle is foundational in the study of optimal control, offering a powerful set of necessary conditions that convert a complex optimization problem into a system of differential equations with boundary conditions. Despite its elegance, the PMP typically yields a two-point boundary value problem, which is analytically intractable for most nonlinear systems and requires numerical techniques such as shooting methods or indirect collocation.

Nevertheless, the PMP provides critical structural insights into the qualitative nature of optimal solutions—such as the emergence of bang-bang or singular arcs—and lays the groundwork for advanced methods in constrained optimization, hybrid systems, and real-time trajectory generation.

Its enduring relevance across disciplines ranging from aerospace engineering to systems biology attests to its fundamental role in modern control theory.

3 Description of the System

The retina is a layered neural tissue at the back of the eye that performs early visual processing. It transforms light into electrical signals via photoreceptors (rods and cones), which then pass through bipolar cells to ganglion cells. Amacrine cells modulate these signals laterally, especially affecting temporal and directional responses. The system can be modeled using a set of ODEs.

3.1 Parameterized and Observed Nonlinear Control Problem

Consider a parameterized nonlinear dynamical system of the form:

$$\begin{cases} \dot{x} = f(x, u, \theta), \\ y = h(x), \end{cases} \quad (1)$$

with fixed initial condition $x(0)$, where:

- $x \in \mathbb{R}^n$ is the state of the system,
- $u \in \mathbb{R}^m$ is the control input,
- $\theta \in \mathbb{R}^p$ is an unknown parameter,
- $y \in \mathbb{R}^q$ is the ideal output assumed to be known,
- f and h are appropriately differentiable functions.

However, in real-world settings, the output is affected by measurement noise $w(t)$, giving rise to:

$$y_{\text{meas}} = h(x(t)) + w(t), \quad (2)$$

where $w(t)$ represents observation noise.

3.2 Parameter Sensitivity

We define the system's sensitivity to the parameters θ using the variable $x_\theta = \frac{\partial x}{\partial \theta}$, which satisfies the following system:

$$\begin{cases} \dot{x}_\theta = \frac{\partial f}{\partial x}(x, u, \theta)x_\theta + \frac{\partial f}{\partial \theta}(x, u, \theta), \\ y_\theta = \frac{\partial h}{\partial \theta}(x) = \frac{\partial h}{\partial x}(x)x_\theta. \end{cases} \quad (3)$$

Here, $y_\theta = \frac{\partial y}{\partial \theta}$ represents the sensitivity of the output with respect to the parameters.

3.3 Quantifying the Deviation Between y and y_{meas}

We aim to quantify the deviation between the ideal output $y(t)$ and the measured output $y_{\text{meas}}(t)$ using the following cost functional:

$$J(\theta, \theta^*) = \int_0^T |y(t) - y_{\text{meas}}(t)|^2 dt \quad (4)$$

where:

- $y(t)$ is the "true" output of the system,
- $y_{\text{meas}}(t) = h(x(t)) + w(t)$ is the measured output, affected by noise $w(t)$.

Let us denote $y^* := h(x)$. Then, the cost functional becomes:

$$J(\theta, \theta^*) = \int_0^T |y(t) - y^*(t) - w(t)|^2 dt \quad (5)$$

By expanding the square and using the linearity of the integral, we get:

$$J(\theta, \theta^*) = \int_0^T |y(t) - y^*(t)|^2 dt - 2 \int_0^T (y(t) - y^*(t))^T w(t) dt + \int_0^T |w(t)|^2 dt \quad (6)$$

Assume that the difference between the estimated parameters and the true parameters $\theta - \theta^*$ is small. We can then expand $y - y^*$ as a Taylor series around θ^* , which gives the following approximation:

$$y - y^* \approx (\theta - \theta^*)^T y_{\theta^*} + \frac{1}{2}(\theta - \theta^*)^T R(\theta - \theta^*) \quad (7)$$

where:

- $y_{\theta^*} = \frac{\partial y}{\partial \theta} \big|_{\theta^*}$ is the sensitivity of the output with respect to the parameters,
- R is a matrix modeling the second-order (quadratic) term.

Neglecting the second-order terms and substituting into equation (6), we obtain:

$$\begin{aligned} J(\theta, \theta^*) = & (\theta - \theta^*)^T \left(\int_0^T y_{\theta^*}(t) y_{\theta^*}(t)^T dt \right) (\theta - \theta^*) \\ & + 2 \left(\int_0^T w(t)^T y_{\theta^*}(t) dt \right) (\theta - \theta^*) \\ & + \int_0^T |w(t)|^2 dt + R' \end{aligned} \quad (8)$$

Define:

$$M(\theta^*) = \int_0^T y_{\theta^*}(t) y_{\theta^*}(t)^T dt \quad \text{and} \quad V(\theta^*) = \int_0^T w(t)^T y_{\theta^*}(t) dt.$$

Then, we can write:

$$J(\theta, \theta^*) = (\theta - \theta^*)^T M(\theta^*) (\theta - \theta^*) + 2V(\theta^*) (\theta - \theta^*) + \int_0^T |w(t)|^2 dt + R' \quad (9)$$

Assuming w is also "small" we get:

$$J(\theta, \theta^*) \approx (\theta - \theta^*)^T M(\theta^*) (\theta - \theta^*) + 2V(\theta^*) (\theta - \theta^*) + \int_0^T |w(t)|^2 dt \quad (10)$$

Now we look to minimize J . The minimum is reached for $M(\theta^*)(\theta - \theta^*) + V(\theta^*) = 0$. We finally obtain :

$$\theta = \theta^* + M^{-1} V \quad (11)$$

4 The Methods

We present different methods for parameter identification for linear systems from scientific literature and a novel method that's currently being developed by the team for experiment design and system identification.

4.1 Method 1: energy constraint

The first method we will look at is the one in [2]. The method will be presented using an example for better clarity. The general principle is the following: We want to minimize the uncertainty of our estimation i.e get as close as possible to the real parameter(s) on average. We want to minimize the Cramer-Rao lower bound which is the inverse of the Fisher Information Matrix (FIM) so we opt for maximization of said FIM and we impose an energy constraint to find the optimal control using the PMP.

4.1.1 Example with One Parameter

Consider the following system:

$$\dot{x} = -x + \theta \cdot u \quad (12)$$

$$y = x + v \quad (13)$$

Where:

- $\dot{x}$ represents the time derivative of x ,
- θ is a constant parameter,
- u is an input, and
- v is a Gaussian white noise with $E(v) = 0$ and $E(v(t)v(\tau)) = \sigma^2\delta(t - \tau)$.

The Information matrix can be shown to be:

$$M = \int_0^T \frac{1}{\sigma^2} x_\theta^2 d\theta \quad (14)$$

Where:

- x_θ is the derivative of x with respect to θ ,
- T is the time horizon,
- σ^2 is the variance of the noise process v .

Let the input be energy-constrained as follows:

$$\int_0^T u^2 dt = E \quad (15)$$

The sensitivity function x_θ satisfies the following differential equation:

$$\dot{x}_\theta = -x_\theta + u \quad (16)$$

4.1.2 Optimization via Pontryagin's Maximum Principle

The maximization of M subject to the constraint (15) is equivalent to the minimization of the performance index:

$$J = -\frac{1}{2} \int_0^T \left(\frac{-x_\theta^2}{\sigma^2} + \left(u^2 - \frac{E}{T} \right) \right) dt \quad (17)$$

The Pontryagin Maximum Principle (PMP) gives the Hamiltonian:

$$\mathcal{H} = \frac{1}{2} \left(\frac{-x_\theta^2}{\sigma^2} + \mu \left(u^2 - \frac{E}{T} \right) \right) + \lambda(-x_\theta + u) \quad (18)$$

where λ is the costate scalar.

The costate equation is:

$$\begin{aligned} \dot{\lambda} &= -\frac{\partial \mathcal{H}}{\partial x_\theta} \\ \dot{\lambda} &= -\lambda + \frac{-x_\theta}{\sigma^2} \end{aligned} \quad (19)$$

The optimal control is obtained from the maximization condition:

$$\frac{\partial \mathcal{H}}{\partial u} = 0$$

which gives:

$$u^* = -\frac{1}{\mu} \lambda \quad (20)$$

The boundary conditions are homogeneous:

$$x_\theta(0) = 0, \quad \lambda(T) = 0$$

4.1.3 Two-Point Boundary Value Problem and Transition Matrix

Substituting u^* into (18), we obtain the two-point boundary value problem:

$$\frac{d}{dt} \begin{bmatrix} x_\theta \\ \lambda \end{bmatrix} = \begin{bmatrix} -1 & -\frac{1}{\mu} \\ \frac{1}{\sigma^2} & 1 \end{bmatrix} \begin{bmatrix} x_\theta \\ \lambda \end{bmatrix} \quad (21)$$

Let $\Phi(T, 0; \mu)$ be the state-transition matrix of (21). Then:

$$\begin{bmatrix} x_\theta(T) \\ \lambda(T) \end{bmatrix} = \begin{bmatrix} \Phi_{xx}(T, 0; \mu) & \Phi_{x\lambda}(T, 0; \mu) \\ \Phi_{\lambda x}(T, 0; \mu) & \Phi_{\lambda\lambda}(T, 0; \mu) \end{bmatrix} \begin{bmatrix} x_\theta(0) \\ \lambda(0) \end{bmatrix} \quad (22)$$

Using the second row of (22) and the boundary condition $\lambda(T) = 0$, we have:

$$\lambda(T) = \Phi_{\lambda\lambda}(T, 0; \mu)\lambda(0) = 0 \quad (23)$$

For a non-trivial solution, the determinant condition must hold:

$$|\Phi_{\lambda\lambda}(T, 0; \mu)| = 0 \quad (24)$$

Equation (24) is the eigenvalue condition for the Hamiltonian system defined by (18). It can be solved numerically, for example using the Newton-Raphson method or a genetic

algorithm. A graph of T can be obtained as a function of μ . By picking the Lagrange multiplier that corresponds to your experiment's length you get the expression of the costate vector and, consequently, the optimal control.

Although this method is accurate, it is costly to compute the solution of (24) for different values of μ especially as the dimension of the problem increases. This brings us to the second method that circumvents these calculations by integrating an augmented system to find the Lagrange multiplier corresponding to the experiment time T

4.2 Method 2: energy constraint with Newton-Raphson method

To try and improve the speed of the calculations an integration method is used [3] instead.

The following system governs blood glucose regulation:

$$\begin{aligned}\dot{x}_1 &= -ax_1 + bx_2, \\ \dot{x}_2 &= -cx_1 - dx_2 + y,\end{aligned}$$

where:

- x_1 is the deviation of the extracellular insulin concentration from the mean (units: units/liter),
- x_2 is the deviation of the extracellular glucose concentration from the mean (units: g/liter),
- y is the rate of glucose intravenous injection (units: g/(liter $\times$ hour)).

The following performance index is used to find the optimal input:

$$M = \int_0^T x_b^2(t) dt, \quad (3)$$

subject to the input energy constraint:

$$E = \int_0^T y^2(t) dt, \quad (4)$$

where $x_b(t) = \frac{\partial x(t)}{\partial b}$.

The optimization problem becomes: Find J where J is

$$J = \max \left(\frac{1}{2} \int_0^T \left[x_b^2(t) - q \left(y^2(t) - \frac{E}{T} \right) \right] dt \right), \quad (6)$$

where q is the Lagrange multiplier.

The vector form of the system is:

$$\dot{x}(t) = Fx(t) + Gy(t), \quad (7)$$

where x is a 2×1 state vector, y is a scalar input, and

$$F = \begin{bmatrix} -a & b \\ -c & -d \end{bmatrix}, \quad G = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (9)$$

And the measurement equation is:

$$z(t) = Hx(t) + v(t), \quad (10)$$

where $z(t)$ is a scalar measurement and $v(t)$ is a zero-mean Gaussian white noise process with

$$E\{v(t)\} = 0, \quad E\{v(t)v^T(\tau)\} = R\delta(t - \tau) = \sigma^2\delta(t - \tau), \quad (11)$$

and measurement matrix

$$H = [0 \quad 1]. \quad (12)$$

Thus the FIM is the following:

$$M = \int_0^T x_b^T H^T R^{-1} H x_b dt, \quad (13)$$

subject to the input energy constraint:

$$\int_0^T y^2 dt = E. \quad (14)$$

We obtain two performance indices with the second showing better convergence [3] :

$$J_1 = \max \left(\frac{1}{2} \int_0^T [x_b^T H^T R^{-1} H x_b - qy^2] dt + q(T) \frac{E}{2} \right), \quad (16)$$

$$J_2 = \max \left(\frac{1}{2} \int_0^T [x_b^T H^T R^{-1} H x_b - qy^2] dt + q(T) \frac{E}{2} - \frac{q^2(0)}{2} \right). \quad (17)$$

The augmented state vector including derivatives and the Lagrange multiplier is the following:

$$X_A = \begin{bmatrix} x_1 \\ x_2 \\ x_{1b} \\ x_{2b} \\ q \end{bmatrix}, \quad (21)$$

$$\dot{X}_A = F_A X_A + G_A y, \quad (22)$$

where

$$F_A = \begin{bmatrix} -a & b & 0 & 0 & 0 \\ -c & -d & 0 & 0 & 0 \\ 0 & 0 & -a & b & 0 \\ 0 & 0 & -c & -d & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad G_A = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}. \quad (23)$$

Thus the hamiltonian function is given by:

$$\mathcal{H} = \frac{1}{2} (-x_A^T H_A^T R^{-1} H_A x_A + K X_A y^2) + p^T (F_A X_A + G_A y), \quad (27)$$

where

$$K = [0 \quad 0 \quad 0 \quad 0 \quad 1]. \quad (28)$$

The PMP provides us with the costate vector equation:

$$\dot{p}(t) = - \left[\frac{\partial \mathcal{H}}{\partial X_A} \right]^T = H_A^T R^{-1} H_A x_A - F_A^T p - K^T y^2/2. \quad (29)$$

The optimal input satisfies:

$$y = -\frac{1}{q} G_A^T p = -\frac{1}{q} p_2. \quad (31)$$

We then get a two-point boundary-value problem:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_{1b} \\ \dot{x}_{2b} \\ \dot{q} \end{bmatrix} = \begin{bmatrix} -a & b & 0 & 0 & 0 \\ -c & -d & 0 & 0 & 0 \\ 0 & 0 & -a & b & 0 \\ 0 & 0 & -c & -d & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_{1b} \\ x_{2b} \\ q \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} y, \quad (32)$$

with boundary conditions:

$$x_1(0) = 0, \quad x_2(0) = 0.1, \quad x_{1b}(0) = 0, \quad x_{2b}(0) = 0, \quad q(0) = q_0, \quad (34)$$

$$p_1(T) = 0, \quad p_2(T) = -\frac{E}{2}, \quad p_3(T) = 0, \quad p_4(T) = 0, \quad p_5(T) = -\frac{E}{2}. \quad (36)$$

Which we can solve using the Newton-Raphson Method:

$$C_{k+1} = C_k - \left(\frac{\partial p(T, C_k)}{\partial C} \right)^{-1} (p(T, C_k) - p_d), \quad (42)$$

where $C = [C_1 \ C_2 \ C_3 \ C_4 \ C_5]^T$.

4.3 The optimization algorithm

A brief explanation of the an optimization algorythm that will be used and compared to the Newton-Raphson Method.

4.3.1 Differential Evolution Algorithm

Differential Evolution (DE) is a stochastic, population-based optimization method designed for continuous-valued, nonlinear, and possibly non-differentiable objective functions. It belongs to the class of evolutionary algorithms and is particularly effective for global optimization over bounded domains.

4.3.2 Overview

Given an objective function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the DE algorithm seeks to find the global minimum over a bounded domain. The algorithm maintains a population of candidate solutions $\{x_i^g\}_{i=1}^{N_p}$ at generation g , where each $x_i^g \in \mathbb{R}^n$, and N_p is the population size.

4.3.3 Algorithm Steps

1. Initialization Each individual is initialized randomly within the search bounds:

$$x_i^0 = x_{\min} + r_i \cdot (x_{\max} - x_{\min}), \quad r_i \sim \mathcal{U}(0, 1)^n$$

2. Mutation For each individual x_i^g , a mutant vector v_i^{g+1} is created using the DE/rand/1 strategy:

$$v_i^{g+1} = x_{r1}^g + F \cdot (x_{r2}^g - x_{r3}^g)$$

where $r1, r2, r3 \in \{1, \dots, N_p\}$, are mutually distinct indices and $\neq i$, and $F \in [0, 2]$ is the mutation factor.

3. Crossover A trial vector u_i^{g+1} is formed by mixing the target vector x_i^g and the donor vector v_i^{g+1} :

$$u_{i,j}^{g+1} = \begin{cases} v_{i,j}^{g+1} & \text{if } r_j \leq CR \text{ or } j = j_{\text{rand}} \\ x_{i,j}^g & \text{otherwise} \end{cases}$$

where $r_j \sim \mathcal{U}(0, 1)$, $CR \in [0, 1]$ is the crossover probability, and $j_{\text{rand}} \in \{1, \dots, n\}$ is a randomly chosen index to ensure at least one component is inherited from v_i^{g+1} .

4. Selection The new individual for the next generation is chosen based on the objective function:

$$x_i^{g+1} = \begin{cases} u_i^{g+1} & \text{if } f(u_i^{g+1}) \leq f(x_i^g) \\ x_i^g & \text{otherwise} \end{cases}$$

4.3.4 Convergence and Characteristics

DE is robust to local minima and does not require gradient information, making it suitable for noisy or discontinuous objective functions. The algorithm balances exploration and exploitation through parameters F , CR , and the chosen mutation strategy. Though convergence guarantees are limited to specific theoretical settings, DE performs well empirically in a wide range of practical optimization problems.

4.3.5 SciPy Implementation Notes

In `scipy.optimize.differential_evolution`, the default strategy is `DE/best/1/bin`, using the current best individual as the base for mutation. The implementation supports box constraints via the `bounds` parameter and optionally handles nonlinear constraints using penalty-based or trust-region methods.

5 Numerical Results and Analysis of the preexisting methods

Most numerical results used Runge-Kutta-4 scheme with a time step of 0.01.

The implementation of the first method using the optimisation algorithm yields a sinusoidal control Figure (1). It hits the wavelength that minimizes the uncertainty. However the control is obtained using the estimation of the true parameter so the wavelength doesn't match further away from the real parameter leading to a sub-optimal optimization. Figure (2) shows the real output and the measured noisy output. Figure (3) shows the control for different values of the lagrange multiplier. The profiles are similar and in fact we can get

a close approximation of the control for a different value of the multiplier by sclaining it [3] Figure (4) is obtained using a bang-bang control instead of the sinus control with the hopes of hitting multiple wavelength at once to increase the odds of having the "best" wavelength in the control. Figure (5) shows the convergence of the inverse analasys algo-rythm in 4 iteration onto the real value of the example from [3] on blood glucose levels. The algorithm shows good convergence for u having 5 to 10 changes. Closer to 10 the convergence is faster. 7 is a good compromise between speed and accuracy.

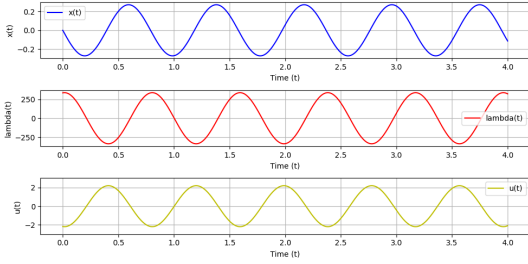


Figure 1: State, costate and control for example 1 with method 1

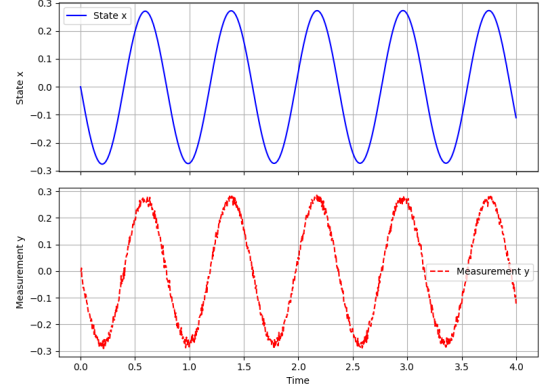


Figure 2: True output and measured noisy output

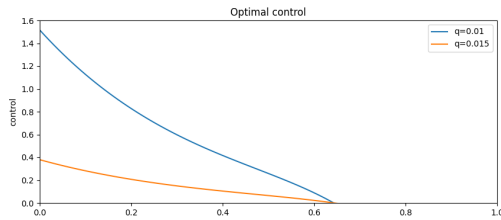


Figure 3: Optimal controls

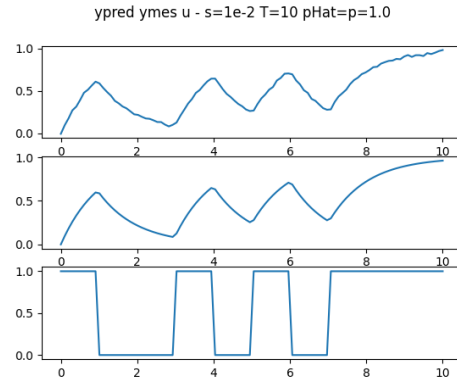


Figure 4: System simulation for a bang-bang control

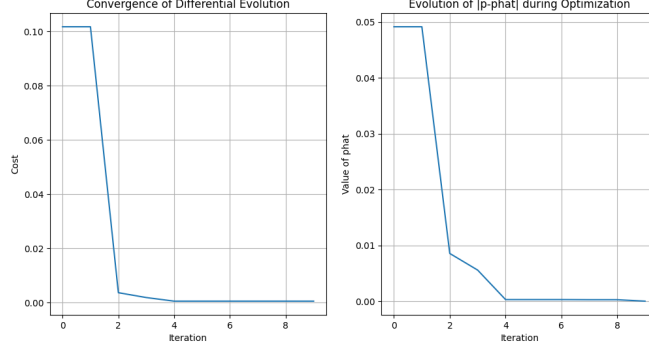


Figure 5: Convergence plots for the differential evolution method

6 Experiment design

In this section we will present the problem and the method to solve it. The general idea is the following: assuming we have a family of inputs, we compute the FIM for all of them (FIM is symmetrical definite-positive). Then we compute the convex hull of the matrices in $\mathbb{R}^d$ which exists since our inputs are continuous functions and we have a measure and a scalar product that's linear with regard to the measure. Finally we use an optimization method to find the optimal input which is situated on the surface on the convex hull and is a convex combination of inputs from the original family.

6.1 Defining the problem

Let $n, m, p, q \in \mathbb{N}$. Let $\Theta \subset \mathbb{R}^p$ be the set of admissible parameters. We set $A : \Theta \rightarrow \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times m}$, $C \in \mathbb{R}^{q \times n}$, and $\sigma \in \mathbb{R}^{n \times n}$. We assume the mapping A to be of regularity C^1 at least. Let $T > 0$ and let $u \in L^\infty([0, T], \mathbb{R}^m)$ (for the presumed Lebesgue measure). Let W be a standard n -dimensional Brownian motion. For a given parameter $\theta \in \Theta$, we consider the controlled stochastic differential equation:

$$dx(t) = A(\theta)x(t) + Bu(t) + \sigma dW(t). \quad (25)$$

Given a deterministic initial condition $x(0) = x_0 \in \mathbb{R}^n$, (25) admits a unique strong solution (admitted), given, for any $t \in [0, T]$, by

$$x(t) = e^{A(\theta)t}x_0 + \int_0^t e^{A(\theta)(t-s)}Bu(s)ds + \int_0^t e^{A(\theta)(t-s)}\sigma dW(s). \quad (26)$$

At any time $t \in [0, T]$, the output of the system is given by $y(t) = Cx(t)$.

Assumption 6.1. We assume there exists $\gamma > 0$ such that for all $\theta \in \Theta$, for any eigenvalue λ of $A(\theta)$, we have

$$\text{Re}(\lambda) < -\gamma.$$

Let $N \in \mathbb{N}$, we set $\Delta = T/N$ and $t_k = k\Delta$, for $k = 0, \dots, N$. We also denote $x_k = x(t_k)$. If we compute increments of x between two dates t_k, t_{k+1} , $0 \leq k < N$, we get:

$$x_{k+1} = e^{A(\theta)\Delta}x_k + \int_{t_k}^{t_{k+1}} e^{A(\theta)(t_{k+1}-s)}Bu(s)ds + \int_{t_k}^{t_{k+1}} e^{A(\theta)(t_{k+1}-s)}\sigma dW(s). \quad (27)$$

Assumption 6.2. There exists a Δ -periodic, essentially-bounded function $U : \mathbb{R} \rightarrow \mathbb{R}^{m \times m}$, and a sequence $(u_k)_{0 \leq k < N} \in (\mathbb{R}^m)^N$ such that:

$$u(t) = U(t)u_k, \quad t \in [t_k, t_{k+1}), 0 \leq k < N. \quad (28)$$

We denote by $\mathcal{U}$ the set of these inputs.

Set

$$F = e^{\Delta A(\theta)}, \quad G = \int_0^\Delta e^{A(\theta)(\Delta-s)} B U(s) ds, \quad W_k = \int_{t_k}^{t_{k+1}} e^{A(\theta)(t_{k+1}-s)} \sigma dW(s).$$

Equation (33) becomes:

$$x_{k+1} = F x_k + G u_k + W_k, \quad (29)$$

with $(W_k)_{0 \leq k < N}$ an i.i.d. sequence of centered Gaussian random vectors with covariance matrix Σ , i.e. $W_k \sim \mathcal{N}(0, \Sigma)$, with:

$$\Sigma = \int_0^\Delta e^{A(\theta)(\Delta-s)} \sigma \sigma^\top e^{A^\top(\theta)(\Delta-s)} ds.$$

Finally, we assume there exists $(\varepsilon_k)_{k < 0 \leq N}$, i.i.d. centered Gaussian random vectors with covariance matrix $\Sigma' \in \mathbb{R}^{q \times q}$, that we assume to be positive-definite, $\varepsilon_k \sim \mathcal{N}(0, \Sigma')$, such that a measurement at t_k , $0 < k \leq N$, follows

$$y_k(u) = \eta(t_k, \theta, u) + \sum_{l=0}^{k-1} C F^{k-l-1} W_l + \varepsilon_k, \quad (30)$$

with η denoting the deterministic output

$$\eta(t_k, \theta, u) = C F^k x_0 + \sum_{l=0}^{k-1} C F^{k-l-1} G u_l. \quad (31)$$

6.2 Computing the information matrix

The crucial observation is that if x is a solution of $\dot{x} = A(\theta)x + Bu$, then, for all $1 \leq i \leq p$,

$$\frac{d}{dt} \partial_{\theta_i} x = \frac{\partial}{\partial \theta_i} \frac{dx}{dt} = \partial_{\theta_i} A(\theta) x + A(\theta) \partial_{\theta_i} x.$$

We may write as well

$$\frac{d}{dt} \begin{pmatrix} x \\ \partial_{\theta_i} x \end{pmatrix} = \bar{A}_i \begin{pmatrix} x \\ \partial_{\theta_i} x \end{pmatrix} + \bar{B} u,$$

letting $\bar{A}_i(\theta) = \begin{pmatrix} A(\theta) & 0 \\ \partial_{\theta_i} A(\theta) & A(\theta) \end{pmatrix}$, $\bar{B} = \begin{pmatrix} B \\ 0 \end{pmatrix}$ and $\bar{C} = (0 \ C)$.

Let $\bar{W}$ be a standard $2n$ -dimensional Brownian motion. With $\bar{x}^i = \begin{pmatrix} x \\ \partial_{\theta_i} x \end{pmatrix}$, the controlled stochastic differential equation becomes:

$$d\bar{x}^i(t) = \bar{A}_i(\theta) \bar{x}^i(t) + \bar{B} u(t) + \begin{pmatrix} \sigma & 0 \\ 0 & 0 \end{pmatrix} d\bar{W}(t) \quad (32)$$

with initial condition $(x_0, 0)$. As before, increments of $\bar{x}^i$ between two dates t_k, t_{k+1} , $0 \leq k < N$, satisfy

$$\bar{x}_{k+1}^i = \bar{F}_i \bar{x}_k^i + \bar{G}_i u_k + \bar{W}_k^i \quad (33)$$

with

$$\bar{F}_i = e^{\Delta \bar{A}_i(\theta)} \quad \bar{G}_i = \int_0^\Delta e^{\bar{A}_i(\theta)(\Delta-s)} \bar{B} U(s) ds, \quad \bar{W}_k^i = \int_{t_k}^{t_{k+1}} e^{\bar{A}_i(\theta)(t_{k+1}-s)} \begin{pmatrix} \sigma & 0 \\ 0 & 0 \end{pmatrix} d\bar{W}(s). \quad (34)$$

The conclusion is that

$$\partial_{\theta_i} \eta(t_k, \theta, u) = \bar{C} \bar{F}_i^k \begin{pmatrix} x_0 \\ 0 \end{pmatrix} + \sum_{l=0}^{k-1} \bar{C} \bar{F}_i^{k-l-1} \bar{G}_i u_l \quad (35)$$

and

$$\partial_{\theta_i} y_k(u) = \partial_{\theta_i} \eta(t_k, \theta, u) + \sum_{l=0}^{k-1} \bar{C} \bar{F}_i^{k-l-1} \bar{W}_l^i. \quad (36)$$

Computationally, we have

$$\bar{F}_i = \begin{pmatrix} F & 0 \\ \partial_{\theta_i} F & F \end{pmatrix}. \quad (37)$$

Set

$$H_i = \text{Diag}(\bar{C}, \dots, \bar{C}) \begin{pmatrix} \text{id}_{\mathbb{R}^{2n \times 2n}} & & & \\ \bar{F}_i & \ddots & & (0) \\ \bar{F}_i^2 & \ddots & \ddots & \\ \vdots & \ddots & \ddots & \ddots \\ \bar{F}_i^{N-1} & \dots & \bar{F}_i^2 & \bar{F}_i & \text{id} \end{pmatrix} \text{Diag}(\bar{G}_i, \dots, \bar{G}_i) \in \mathbb{R}^{Nq \times Nm}, \quad (38)$$

and

$$K_i = \begin{pmatrix} \bar{C} \bar{F}_i \\ \bar{C} \bar{F}_i^2 \\ \vdots \\ \bar{C} \bar{F}_i^N \end{pmatrix} \begin{pmatrix} \text{id} \\ 0_{\mathbb{R}^{n \times n}} \end{pmatrix} \in \mathbb{R}^{Nq \times n} \quad (39)$$

Denoting $\mathbf{u} \in \mathbb{R}^{Nm}$ the vector concatenation of $u_0, \dots, u_{N-1}$, that is, $\mathbf{u}^\top = (u_0^\top \quad \dots \quad u_{N-1}^\top)$,

$$\partial_{\theta_i} \boldsymbol{\eta}(\theta, u) = H_i \mathbf{u} + K_i x_0. \quad (40)$$

Then we get the full expression of the Fisher information matrix $\mathbf{M}$:

$$\mathbf{M}_{ij}(\theta, \delta_u) = (H_i \mathbf{u} + K_i x_0)^\top \boldsymbol{\Sigma}^{-1} (H_j \mathbf{u} + K_j x_0) + Q_{ij}. \quad (41)$$

In particular setting $L_{ij} = (H_i^\top \boldsymbol{\Sigma}^{-1} H_j + H_j^\top \boldsymbol{\Sigma}^{-1} H_i)/2$ and $x_0 = 0$, we finally get

$$\mathbf{M}_{ij}(\theta, \delta_u) = \mathbf{u}^\top L_{ij} \mathbf{u} + Q_{ij}. \quad (42)$$

7 Explanation of the Python Script (tools.py)

This Python script implements several numerical methods for solving differential equations and manipulating matrix-based models. Below is a detailed breakdown of each function.

7.1 Function Overview

The script consists of several key functions that solve differential equations and perform matrix manipulations, using numerical integration (Runge-Kutta 4th order) and matrix operations. These functions are organized into different sections based on their purpose.

7.2 Functions for Solving Differential Equations

7.2.1 `getG(f, n, m, t0, tf, dt)`

This function solves a system of differential equations using the Runge-Kutta 4th order (RK4) method. It computes the time evolution of a matrix G over the interval $[t_0, t_f]$.

- **Parameters:**

- f : The function representing the system of differential equations.
- n, m : The dimensions of the matrix G to be solved.
- t_0, t_f : The initial and final times of the integration.
- dt : The time step for the integration.

- **Process:** This function uses the RK4 method to iteratively compute the matrix G at each time step. The RK4 method requires four intermediate evaluations (k_1, k_2, k_3, k_4) of f at different points within the integration interval, which are then combined with appropriate weights to compute the next value of G .

- **Output:** The final matrix G after the full time span is returned.

7.2.2 `getGi(f, n, m, t0, tf, dt)`

This function is similar to `getG`, but instead computes a matrix G_i with different initial conditions. The integration process follows the same RK4 scheme, with G_i initialized as a $2n \times m$ zero matrix.

7.2.3 `getSigma(f, n, t0, tf, dt)`

This function computes a matrix Σ by solving a differential equation using RK4. The matrix is initialized as an $n \times n$ zero matrix and updated over time.

7.2.4 `getSi(f, n, t0, tf, dt)`

This function computes a matrix S_i , similarly to `getSigma`, but initialized as a $2n \times 2n$ zero matrix and updated via RK4 integration.

7.3 Matrix Manipulation Functions

7.3.1 `getSigmaBold( $\Sigma$ , N, F, C, lsigmap)`

This function computes a bold version of the covariance matrix:

$$\Sigma = C \left(\sum_{k=0}^{N-1} F^k \Sigma F^{kT} \right) C^T + \text{diag}(lsigmap)$$

It iterates powers of F and accumulates transformed versions of Σ , adding a regularization term $lsigmap$ on the diagonal.

7.3.2 `getH1H2(N, Fp1, Fp2, Gp1, Gp2, Cp)`

This function computes two bold matrices H_1 and H_2 by constructing block-structured matrices using powers of F_{p1} and F_{p2} , combined with G_{p1}, G_{p2}, Cp , and then assembled into 10×10 arrays.

7.3.3 `getSigmaiBold(C, Cp, newCp, newCpT, Sigma1, n, N, Fp1)`

This computes another covariance matrix:

$$\Sigma_1 = Cp \left(\sum_k F_p^k \Sigma_1 F_p^{kT} \right) newCp^T + newCp \left(\sum_k F_p^k \Sigma_1 F_p^{kT} \right) Cp^T$$

for appropriate powers of F_{p1} .

7.3.4 `getQ( $\Sigma^{-1}$ , $\Sigma_1$ , $\Sigma_2$ )`

Computes a 2×2 matrix Q whose entries are half the trace of products like $\Sigma^{-1} \Sigma_1 \Sigma^{-1} \Sigma_1$, measuring similarity between the bold covariance matrices.

7.3.5 `getListM(H1, H1T, H2, H2T, $\Sigma^{-1}$ , array_list, Q)`

Constructs a list of bold matrices M_{ij} by combining H_1, H_2 , their transposes, and Σ^{-1} . For each vector in 'array_list', it computes a 2×2 matrix:

$$M_{ij} = \mathbf{u}^\top L_{ij} \mathbf{u} + Q$$

7.3.6 `getListMxyz(H1, H1T, H2, H2T, SigmaBoldinv, array-list, Q)`

A variant of `getListM` that returns a different representation (e.g. a 3D-point version) of the bold matrices for further geometric analysis.

7.4 Auxiliary geometric functions

7.4.1 `prodscalm2D(M, N)`

Computes the scalar (dot) product of two 2×2 matrices M and N as a sum of element-wise products.

7.4.2 `calculate_matrix(M)`

This function converts a 3D point $M = (x, y, z)$ into a 2×2 matrix:

$$\begin{bmatrix} z - y & x \\ x & z + y \end{bmatrix}$$

7.4.3 log_det_maximum(M1, M2, M3, num_points)

This function explores convex combinations of three 3D points $M1, M2, M3$ on a sampling grid of size ‘num_points’, *mapseachcombinationtoamatrixvia‘calculate_matrix’, computes its determinant along with weights* (w_1, w_2, w_3) .

7.4.4 face(m1, m2, m3, p)

This version of ‘face’ evaluates a selection criterion based on three 3D points mapped to matrices via ‘calculate-matrix’, then computes inverse, log-determinant, and scalar products to produce a final scalar value. It returns the minimum among these criteria.

7.5 Optimal control computation

7.5.1 getOptimalControl(p1, p2)

This high-level routine computes optimal controls given parameter inputs p_1, p_2 . It:

- Constructs system matrices $A = A_0 + p_1 A_1 + p_2 A_2$,
- Computes exponentials F, F_{p1}, F_{p2} ,
- Solves for $G, \Sigma, \Sigma_1, \Sigma_2$ via RK4 routines,
- Builds bold covariance matrices using projection and transformations,
- Extracts bold matrices H_1, H_2 , inverse covariance, and matrix Q ,
- Builds a convex hull of resulting points, evaluates log-determinant criteria, and selects optimal weights,
- **Outputs:** Optimal control vectors u_1, u_2, u_3 , weights w_1, w_2, w_3 , and matrices $\Sigma, \Sigma^{-1}, \Sigma_1, \Sigma_2$.

7.6 Additional functions

7.6.1 gety(truep1, truep2, control)

Simulates the system output y over time given true parameters p_1, p_2 and a control sequence, by integrating noise-driven dynamics and applying the control.

7.6.2 getetha(p1, p2, control)

Computes intermediate signals η , along with derivatives $\frac{\partial \eta}{\partial p_1}$ and $\frac{\partial \eta}{\partial p_2}$, used for sensitivity or gradient-based optimization.

7.6.3 getJ(y, etha, Σ, Σ^{-1})

Computes an objective (e.g. negative log-likelihood) by combining the squared error between y and η weighted by Σ^{-1} , plus the log-determinant term $\log \det \Sigma$.

7.6.4 getJp1Jp2(y, etha, dethap1, dethap2, $\Sigma^{-1}, \Sigma_1, \Sigma_2$)

Computes gradients of the objective with respect to parameters p_1 and p_2 , using trace and quadratic forms involving the bold covariance matrices.

8 Results

The following pairs of Figures show the convex hull for different control families with the simulation of the system response using the optimal control from each family. The FIM corresponding to the optimal control is the dot in red. Figure(6)-(7) show the plots for a piece-wise constant control and a maximum $\text{tr}(\text{FIM})$ of about 150 and a sytem response ranging from -1.5 to 1.5. maximizing the trace $\text{tr}(\text{FIM})$ and the $\log(\det(\text{FIM}))$ give similar results [2]. The t^2 control Figure(8)-(9) has a worse $\log\det(\text{FIM})$ and the system response range is lower. The $4t + \sin(t) + \cos(2t)$ control family shows the best results Figure(10)-(11) with a wide response range with $(t^4 - 2t)/(t + 1)$ coming in third place Figure(12)-(13). Finnaly Figure(14) shows Figure(6) with the plot of the SPD matrices cone with the convex hull being above the cone which is to be expected as FIM should be SPD.

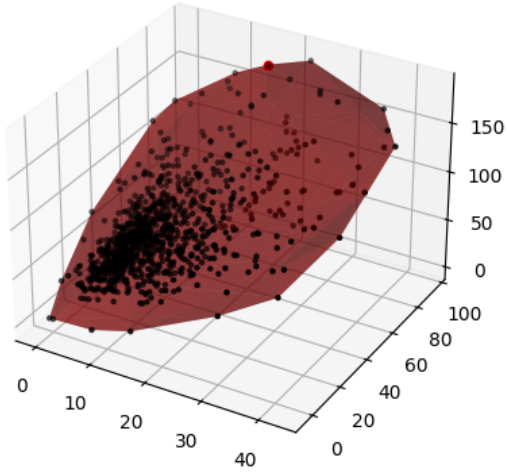


Figure 6: Convexe hull for piece-wise constant control.

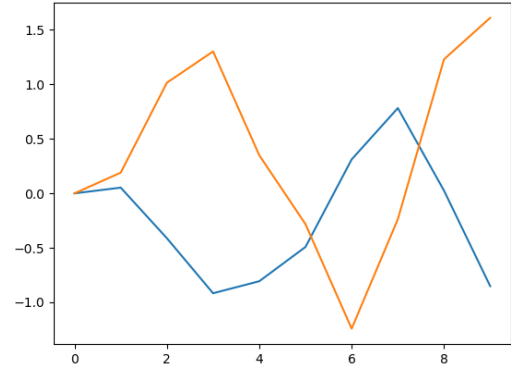


Figure 7: Simulation with the optimal piece-wise constant control.

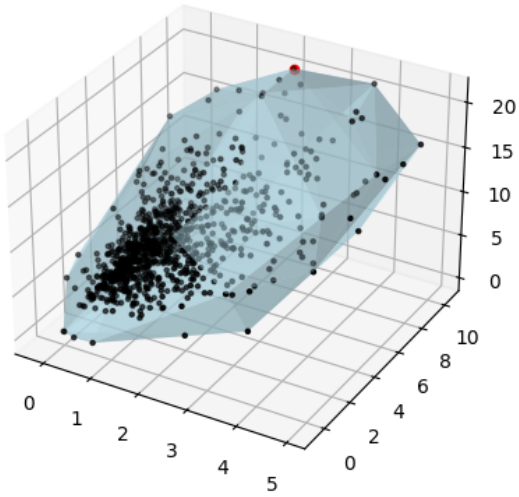


Figure 8: Convexe hull for t^2 control.

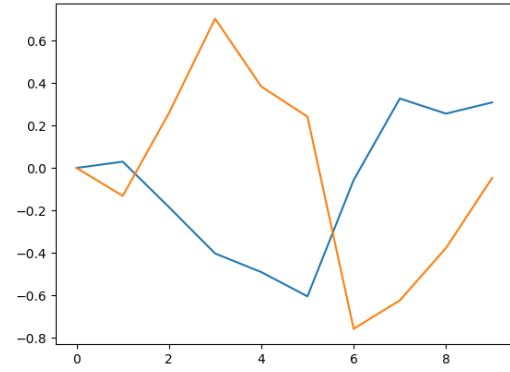


Figure 9: Simulation with the optimal t^2 control.

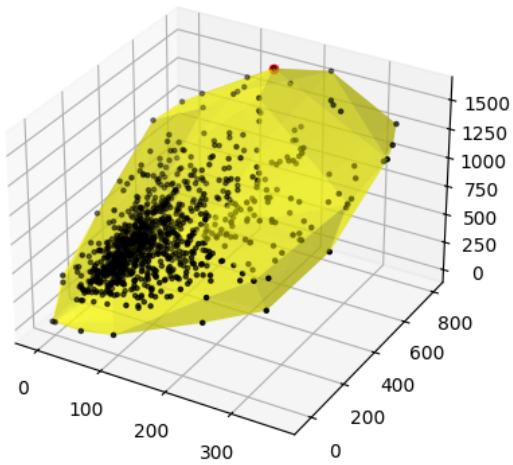


Figure 10: Convexe hull for $4t + \sin(t) + \cos(2t)$ control.

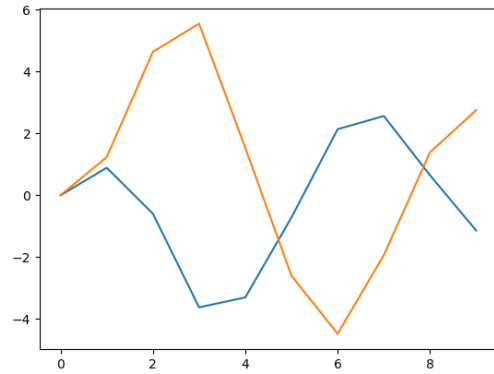


Figure 11: Simulation with the optimal control.

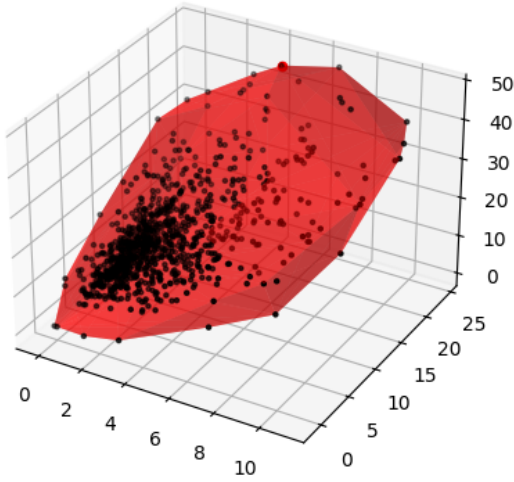


Figure 12: Convex hull for $(t^4 - 2t)/(t + 1)$ control.

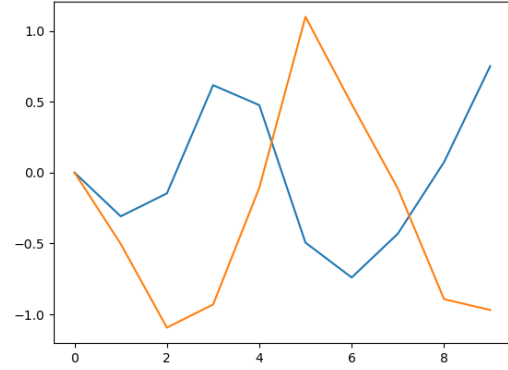


Figure 13: Simulation with the optimal control.

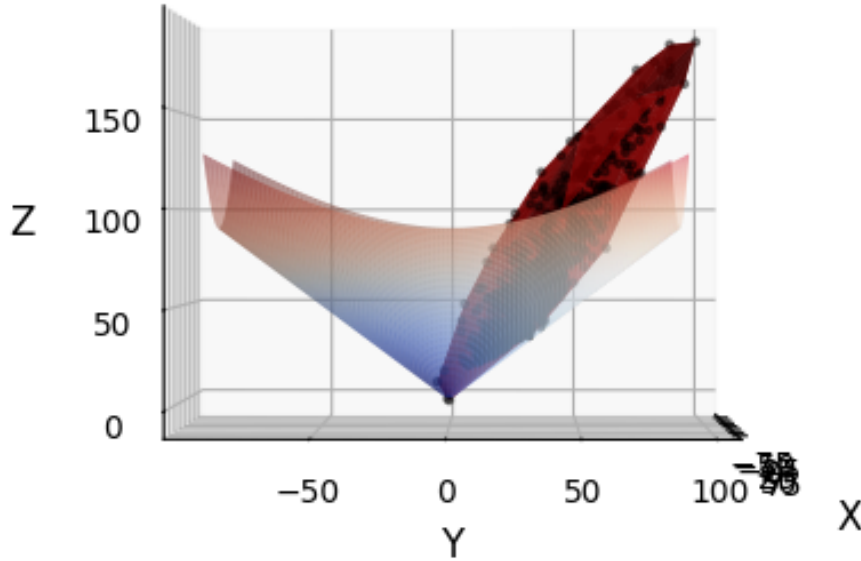


Figure 14: The convex hull with the cone of SPD matrices.

9 Conclusion

To summarize, during my internship I studied the state of art optimal control method for experiment design and helped develop a python script for a novel method for designing an experiment. I also learned a lot about research in a multidisciplinary environment. A possible continuation would be to improve the efficiency of the code and to implement an inverse analysis using the optimal control and to compare it to sub-optimal ones.

References

- [1] E. Trélat , *Contrôle optimal : théorie et applications*, octobre 2023.
- [2] R. K. Mehra , *Optimal Inputs for Linear System Identification*, IEEE TRANSACTIONS ON AUTOMATIC CONTROL, VOL. AC-19,NO. 3,JUNE 1974.
- [3] R. E. Kabala, K. Spingarn *Optimal Inputs for Blood Glucose Regulation Parameter Estimation, Part 3* JOURNAL OF OPTIMIZATION THEORY AND APPLICATIONS:Vol. 33, No. 2, FEBRUARY 1981
- [4] . Storn and K. Price, “Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces”, Journal of Global Optimization, 1997.

10 Annexes

10.1 Codes sources

Voici les codes Python de la première méthode :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.integrate import solve_ivp
4 from scipy.optimize import differential_evolution
5 from scipy.linalg import expm
6
7
8
9 # Param tres
10 sigma2 = 0.0001
11 p0 = 0. # Valeur initiale de p
12 #mu0 = 150
13
14 def M(mu):
15     mu = np.squeeze(mu)
16     # D finir l'EDO
17     def edo(t, p, mu, sigma2):
18         return - p * p * (1./sigma2) - 2. * p - 1. / mu
19     # Intervalle de temps
20     t_span = (0, .5)
21     t_eval = np.linspace(t_span[0], t_span[1], 1000000)
22     sol = solve_ivp(edo, t_span, [p0], args=(mu, sigma2), t_eval=t_eval
23 )
24
25     # Trouver le moment o p(t) descend en dessous de -60
26     t_crit = None
27     for i in range(1, len(sol.t)):
28         if np.abs(sol.y[0][i]) > 100:
29             t_crit = sol.t[i]
30             break
31
32     T = t_crit
33
34     A = np.array([[-1, -1/mu],
35                  [1/sigma2, 1]])
36
37     # Calcul de exp(AT)
38     exp_AT = expm(A * T)
39     # print("La matrice exp(AT) est :")
40     # print(exp_AT)
41     # print(np.sqrt(mu*mu/t_crit))
42     # Extraire l' lment en bas gauche (indice [1,0])
43     lower_left_value = exp_AT[1, 1]
44     return np.abs(lower_left_value)
45
46 # Define bounds for mu (adjust as needed)
47 bounds = [(153, 157)] # Example bounds: mu between 1 and 200
48
49 # Perform differential evolution to minimize M
50 result = differential_evolution(M, bounds, strategy='best1bin', maxiter
51                                =5, popsize=5, tol=0.01, mutation=(0.5, 1), recombination=0.7)
```

```

51 print("Optimization result:")
52 print(f"Optimal mu: {result.x[0]}")
53 print(f"Minimum M value: {result.fun}")
54
55 import numpy as np
56 import matplotlib.pyplot as plt
57
58 mu = 155.
59 sigma2 = 0.0001
60
61 # Define the system matrix A
62 A = np.array([[ -1., -1./mu],
63               [1./sigma2, 1.]])
64
65 # Initial conditions
66 x0 = 0
67 lambda0 = 336.0381084798674
68 y0 = np.array([x0, lambda0])
69
70 # Time step and number of steps
71 t_max = 4
72 dt = 0.001*t_max
73 steps = int(t_max / dt)
74
75 # Arrays to store the results
76 time = np.linspace(0, t_max, steps)
77 x_values = []
78 lambda_values = []
79 u_values = []
80
81 # Runge-Kutta 4th order method loop
82 y = y0
83 for t in time:
84     x_values.append(y[0])
85     lambda_values.append(y[1])
86     u_values.append(-y[1]/mu)
87     # Calculate the Runge-Kutta coefficients
88     k1 = dt * np.dot(A, y)
89     k2 = dt * np.dot(A, y + k1 / 2)
90     k3 = dt * np.dot(A, y + k2 / 2)
91     k4 = dt * np.dot(A, y + k3)
92
93     # Update y using the weighted average of slopes
94     y = y + (1/6) * (k1 + 2*k2 + 2*k3 + k4)
95
96
97
98 # Plot the results
99 plt.figure(figsize=(10, 5))
100
101 # Plot x(t) and lambda(t)
102 plt.subplot(3, 1, 1)
103 plt.plot(time, x_values, label='x(t)', color='b')
104 plt.xlabel('Time (t)')
105 plt.ylabel('x(t)')
106 #plt.ylim(-10,10)
107 plt.grid(True)
108 plt.legend()

```

```

109
110 plt.subplot(3, 1, 2)
111 plt.plot(time, lambda_values, label='lambda(t)', color='r')
112 plt.xlabel('Time (t)')
113 plt.ylabel('lambda(t)')
114 #plt.ylim(-10,10)
115 plt.grid(True)
116 plt.legend()
117
118 plt.subplot(3, 1, 3)
119 plt.plot(time, u_values, label='u(t)', color='y')
120 plt.xlabel('Time (t)')
121 plt.ylabel('u(t)')
122 plt.ylim(-3,3)
123 plt.grid(True)
124 plt.legend()
125
126 plt.tight_layout()
127 plt.show()
128
129 from experienceAndMesurement import *
130
131 np.random.seed(10571) # Ensures reproducibility of random results
132 npoints=1000
133 # Create time points
134 tpoints = constructTime(t0=0, tmax=t_max, npoints=npoints)
135
136 # Generate bang-bang control (5 ones, 5 negative ones repeating)
137 control = np.array(u_values)
138
139 # Run the experiment
140 x, y = experiment(f=-1., h=1.0, sigma2=sigma2, x0=0., u=control,
141                 tpoints=tpoints)
142
143 # Create subplots
144 fig, axes = plt.subplots(2, 1, figsize=(8, 6), sharex=True)
145
146 # Plot state x
147 axes[0].plot(tpoints, x, label="State x", color="b")
148 axes[0].set_ylabel("State x")
149 axes[0].legend()
150 axes[0].grid(True)
151
152 # Plot measurement y
153 axes[1].plot(tpoints, y, label="Measurement y", color="r", linestyle="
    dashed")
154 axes[1].set_xlabel("Time")
155 axes[1].set_ylabel("Measurement y")
156 axes[1].legend()
157 axes[1].grid(True)
158
159 # Adjust layout for better spacing
160 plt.tight_layout()
161
162 # Show the plots
163 plt.show()

```

Voici les code Python de la deuxième méthode :

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Created on Wed Apr  9 10:23:48 2025
5
6  @author: Mohamad SAMMAN
7
8  2D System:  $\dot{x} = Ax + Bu$ ,  $y = Cx$ ,  $y_{mes} = y + v$ 
9  Optimization of parameter  $p$  in  $B = \begin{bmatrix} p \\ 0 \end{bmatrix}$ 
10 """
11
12 import numpy as np
13 from scipy.optimize import differential_evolution as difev
14 import matplotlib.pyplot as plt
15 from datetime import datetime
16
17 # Parameters
18 tstart = 0.
19 tend = 1.02
20 n = 100
21 s = 1.e-2
22 tt = np.linspace(tstart, tend, n)
23 dt = tt[1] - tt[0]
24
25 # Initialize signals
26 x = np.zeros((n, 2))           # 2D state
27 xhat = np.zeros((n, 2))        # Estimated state
28 y = np.zeros(n)                # true output
29 ypred = np.zeros(n)            # predicted output
30 ymes = np.zeros(n)             # measured output
31 u = np.ones(n)                 # control input
32
33 # Noise
34 np.random.seed(40)
35 v = np.random.normal(0, s, n)
36
37 # Piecewise constant control
38 nu = 10
39 tu = np.linspace(tstart, tend, nu + 1)
40 cst1 = 2*np.sqrt(0.04894)
41 uval = np.random.choice([0, cst1], size=nu, p=[0.5, 0.5])
42 for i in range(nu):
43     u[(tt >= tu[i]) & (tt < tu[i + 1])] = uval[i]
44
45
46 # --- Cost function ---
47 def func(phat):
48     phat = float(np.squeeze(phat))
49     p = 0.208
50
51     # System dynamics
52     A = np.array([[0.78, 0.208],
53                   [4.34, 2.92]])
54     B = np.array([0.0, 1.0])
55     C = np.array([0.0, 1.]) # observe only x1
56
57     AHat = np.array([[0.78, phat],
58                     [4.34, 2.92]])

```

```

59     BHat = B
60
61     # Initial condition
62     x_sim = np.zeros((n, 2))
63     xhat_sim = np.zeros((n, 2))
64     x0 = np.array([0.0, 0.0])
65     x_sim[0] = x0
66     xhat_sim[0] = x0
67
68     y[0] = (C @ x0).item()
69
70     # True system simulation
71     for t in range(n - 1):
72         u_t = u[t]
73         u_t1 = u[t + 1]
74         u_avg = 0.5 * (u_t + u_t1)
75
76         k1 = dt * (A @ x_sim[t] + B.flatten() * u_t)
77         k2 = dt * (A @ (x_sim[t] + k1 / 2) + B.flatten() * u_avg)
78         k3 = dt * (A @ (x_sim[t] + k2 / 2) + B.flatten() * u_avg)
79         k4 = dt * (A @ (x_sim[t] + k3) + B.flatten() * u_t1)
80
81         x_sim[t + 1] = x_sim[t] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
82         y[t + 1] = (C @ x_sim[t + 1]).item()
83
84     # Predicted system simulation
85     for t in range(n - 1):
86         u_t = u[t]
87         u_t1 = u[t + 1]
88         u_avg = 0.5 * (u_t + u_t1)
89
90         k1 = dt * (AHat @ xhat_sim[t] + BHat.flatten() * u_t)
91         k2 = dt * (AHat @ (xhat_sim[t] + k1 / 2) + BHat.flatten() *
92 u_avg)
93         k3 = dt * (AHat @ (xhat_sim[t] + k2 / 2) + BHat.flatten() *
94 u_avg)
95         k4 = dt * (AHat @ (xhat_sim[t] + k3) + BHat.flatten() * u_t1)
96
97         xhat_sim[t + 1] = xhat_sim[t] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
98         ypred[t + 1] = (C @ xhat_sim[t + 1]).item()
99
100     ymes[:] = y + v
101     cost = np.sum((ymes - ypred) ** 2)
102     return cost
103
104 # Optimization tracking
105 history = []
106 phat_history = []
107
108 def callback(xk, convergence):
109     cost = func(xk[0])
110     history.append(cost)
111     phat_history.append(xk[0])
112
113 # Optimize using differential evolution

```

```

115 result = difev(func, bounds=[(0., 1.)], strategy='best1bin',
116                 maxiter=100, popsize=5, tol=1e-8,
117                 mutation=(0.5, 1), recombination=0.7,
118                 callback=callback)
119
120 phatopt = f'Optimal value of p (phat): {result.x[0]}'
121 costopt = f'Minimum cost: {result.fun}'
122
123 print(phatopt)
124 print(costopt)
125
126 # Plot convergence
127 plt.figure(figsize=(12, 6))
128 plt.subplot(1, 2, 1)
129 plt.plot(history)
130 plt.xlabel('Iteration')
131 plt.ylabel('Cost')
132 plt.title('Convergence of Differential Evolution')
133 plt.grid(True)
134
135 plt.subplot(1, 2, 2)
136 plt.plot(np.abs(result.x[0]-phat_history))
137 plt.xlabel('Iteration')
138 plt.ylabel('Value of phat')
139 plt.title('Evolution of phat during Optimization')
140 plt.grid(True)
141
142 current_time = datetime.now().strftime('%Y-%m-%d_%H-%M-%S')
143 filename = f'Plots/2Dtest1_{current_time}_phat_{result.x[0]:.4f}_cost_{
144             result.fun:.4f}.png'
145 #plt.savefig(filename)
146 plt.show()

```

Voici le code Python de la boîte à outils utilisé pour la deuxième partie :

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import scipy as sp
4 import itertools
5 import math
6 from tools import *
7 from scipy.spatial import ConvexHull
8 from mpl_toolkits.mplot3d import Axes3D
9 from matplotlib import cm
10 from matplotlib.colors import LightSource
11
12 def getG(f,n,m,t0,tf,dt):
13     NN = int((tf-t0) / dt) + 1 #nombre de point de discretion
14     G_rk4 = np.zeros((n, m)) #initialisation de Gi(0) = 0
15     t=t0
16     for _ in range(NN):
17
18         k1 = f(G_rk4,t)
19         k2 = f(G_rk4 + (dt / 2) * k1,t)
20         k3 = f(G_rk4 + (dt / 2) * k2,t)
21         k4 = f(G_rk4 + dt * k3,t)
22
23         G_rk4 = G_rk4 + (dt / 6) * (k1 + 2*k2 + 2*k3 + k4)
24         t=t+dt

```

```

25
26     G=G_rk4
27     return(G)
28
29
30 def getGi(f,n,m,t0,tf,dt):
31     NN = int((tf-t0) / dt) + 1 #nombre de point de discretion
32     G_rk4 = np.zeros((2*n, m)) #initialisation de Gi(0) = 0
33     t=t0
34     for _ in range(NN):
35
36         k1 = f(G_rk4,t)
37         k2 = f(G_rk4 + (dt / 2) * k1,t)
38         k3 = f(G_rk4 + (dt / 2) * k2,t)
39         k4 = f(G_rk4 + dt * k3,t)
40
41         G_rk4 = G_rk4 + (dt / 6) * (k1 + 2*k2 + 2*k3 + k4)
42         t=t+dt
43
44     Gi=G_rk4
45     return(Gi)
46
47 def getSigma(f,n,t0,tf,dt):
48     NN = int((tf-t0) / dt) + 1 #nombre de point de discretion
49     S_rk4 = np.zeros((n, n)) #initialisation de Gi(0) = 0
50
51     for _ in range(NN):
52
53         k1 = f(S_rk4)
54         k2 = f(S_rk4 + (dt / 2) * k1)
55         k3 = f(S_rk4 + (dt / 2) * k2)
56         k4 = f(S_rk4 + dt * k3)
57
58         S_rk4 = S_rk4 + (dt / 6) * (k1 + 2*k2 + 2*k3 + k4)
59
60     Sigma = S_rk4
61     return(Sigma)
62
63
64 def getSi(f,n,t0,tf,dt):
65     NN = int((tf-t0) / dt) + 1 #nombre de point de discretion
66
67     Si_rk4 = np.zeros((2*n, 2*n))
68
69     for _ in range(NN):
70
71         k1 = f(Si_rk4)
72         k2 = f(Si_rk4 + (dt / 2) * k1)
73         k3 = f(Si_rk4 + (dt / 2) * k2)
74         k4 = f(Si_rk4 + dt * k3)
75
76         Si_rk4 = Si_rk4 + (dt / 6) * (k1 + 2*k2 + 2*k3 + k4)
77
78     Si = Si_rk4
79     return(Si)
80
81 def getSigmaBold(Sigma,N,F,C,lsigmap):
82     list_Sigma_l=[]

```



```

83     list_puissances_F=[]
84
85     Sigma_l_temp = Sigma
86     puissance_F_temp = np.eye(2)
87
88     for _ in range(N):
89         list_puissances_F.append(puissance_F_temp)
90         list_Sigma_l.append(Sigma_l_temp)
91
92         puissance_F_temp = puissance_F_temp@F
93         Sigma_l_temp = Sigma_l_temp + puissance_F_temp @ Sigma @
puissance_F_temp.T
94
95
96     CT=np.transpose(C)
97
98
99     SigmaBold = np.zeros((10,10))
100
101     array_10x10 = []
102
103     for i in range(1,11):
104         row = []
105         for j in range(1,11):
106             triplet = [max(i - j, 0), min(i, j), max(j - i, 0)]
107             row.append(triplet)
108             array_10x10.append(row)
109
110
111     for i in range(N):
112         for j in range(N):
113             SigmaBold[i,j] = C@((list_puissances_F[array_10x10[i][j]
][0]))@((list_Sigma_l[array_10x10[i][j][1]-1])@((list_puissances_F[
array_10x10[i][j][2])).T))@CT
114             if(i==j):
115                 SigmaBold[i,j] = SigmaBold[i,j] + lsignap
116
117     return(SigmaBold)
118 #####
119 def getH1H2(N,Fp1,Fp2,Gp1,Gp2,Cp):
120     list_puissances_Fp1=[]
121     list_puissances_Fp2=[]
122
123     puissance_Fp1_temp = np.eye(4)
124     puissance_Fp2_temp = np.eye(4)
125
126
127     for _ in range(N):
128         list_puissances_Fp1.append(puissance_Fp1_temp)
129         list_puissances_Fp2.append(puissance_Fp2_temp)
130
131         puissance_Fp1_temp = puissance_Fp1_temp@Fp1
132         puissance_Fp2_temp = puissance_Fp2_temp@Fp2
133
134     listH1machin=[]
135     listH2machin=[]
136     for _ in range(N):
137         listH1machin.append(Cp@list_puissances_Fp1[_]@Gp1)

```

```

138         listH2machin.append(Cp@list_puissances_Fp2[_]@Gp2)
139
140     H1 = np.zeros((10,10))
141     H2 = np.zeros((10,10))
142
143     for j in range(10):
144         for i in range(10-j):
145             H1[i+j,i]=listH1machin[j]
146             H2[i+j,i]=listH2machin[j]
147
148
149     return(H1,H2)
150
151
152 def getSigmaiBold(C,Cp,newCp,newCpT,Sigma1,n,N,Fp1):
153     list_Sigma1_l=[]
154     list_puissances_Fp1=[]
155
156     Sigma1_l_temp = Sigma1
157     puissance_Fp1_temp = np.eye(2*n)
158
159     for _ in range(N):
160         list_puissances_Fp1.append(puissance_Fp1_temp)
161         list_Sigma1_l.append(Sigma1_l_temp)
162
163         puissance_Fp1_temp = puissance_Fp1_temp@Fp1
164         Sigma1_l_temp = Sigma1_l_temp + puissance_Fp1_temp @ Sigma1 @ (
165             puissance_Fp1_temp.T)
166
167     Sigma1Bold = np.zeros((10,10))
168
169     CT=np.transpose(C)
170     CpT=Cp.T
171
172     array_10x10 = []
173
174     for i in range(1,11):
175         row = []
176         for j in range(1,11):
177             triplet = [max(i - j, 0), min(i, j), max(j - i, 0)]
178             row.append(triplet)
179         array_10x10.append(row)
180
181
182
183     for i in range(N):
184         for j in range(N):
185             Sigma1Bold[i,j] = (Cp@((list_puissances_Fp1[array_10x10[i][j]
186                 ][0]))@((list_Sigma1_l[array_10x10[i][j][1]-1])@((
187                 list_puissances_Fp1[array_10x10[i][j][2]]) .T))@newCpT) + newCp@((
188                 list_puissances_Fp1[array_10x10[i][j][0]))@((list_Sigma1_l[
189                 array_10x10[i][j][1]-1])@((list_puissances_Fp1[array_10x10[i][j]
190                 ][2])) .T))@CpT
191
192     SigmaiBold=Sigma1Bold
193     return(SigmaiBold)

```

```

190
191 def getQ(SigmaBoldinv, Sigma1Bold, Sigma2Bold):
192     Q11 = 0.5 * np.trace(SigmaBoldinv @ Sigma1Bold @ SigmaBoldinv @
193                           Sigma1Bold)
194     Q12 = 0.5 * np.trace(SigmaBoldinv @ Sigma1Bold @ SigmaBoldinv @
195                           Sigma2Bold)
196     Q22 = 0.5 * np.trace(SigmaBoldinv @ Sigma2Bold @ SigmaBoldinv @
197                           Sigma2Bold)
198     Q = np.zeros((2,2))
199     Q[0,0] = Q11
200     Q[0,1] = Q12
201     Q[1,0] = Q12
202     Q[1,1] = Q22
203     return(Q)
204
205 def getListM(H1, H1T, H2, H2T, SigmaBoldinv, array_list, Q):
206     M11 = 0.5*(H1T@SigmaBoldinv@H1 + H1T@SigmaBoldinv@H1)
207     M12 = 0.5*(H1T@SigmaBoldinv@H2 + H2T@SigmaBoldinv@H1)
208     M21 = 0.5*(H2T@SigmaBoldinv@H1 + H1T@SigmaBoldinv@H2)
209     M22 = 0.5*(H2T@SigmaBoldinv@H2 + H2T@SigmaBoldinv@H2)
210
211     MBoldij_list = []
212
213     MBold11, MBold12, MBold21, MBold22 = 0., 0., 0., 0.
214     #print(np.squeeze(array_list[0]))
215     MBoldij_temp = np.zeros((2,2))
216
217     MBoldij_list_x_y_z = []
218
219     for i in range(len(array_list)):
220         MBold11 = np.transpose(np.squeeze(array_list[i]))@M11@np.
221         squeeze(array_list[i])
222         MBold12 = np.transpose(np.squeeze(array_list[i]))@M12@np.
223         squeeze(array_list[i])
224         MBold21 = np.transpose(np.squeeze(array_list[i]))@M21@np.
225         squeeze(array_list[i])
226         MBold22 = np.transpose(np.squeeze(array_list[i]))@M22@np.
227         squeeze(array_list[i])
228         MBoldij_temp[0,0] = MBold11 + Q[0,0]
229         MBoldij_temp[0,1] = MBold12 + Q[0,1]
230         MBoldij_temp[1,0] = MBold21 + Q[1,0]
231         MBoldij_temp[1,1] = MBold22 + Q[1,1]
232         MBoldij_list.append(MBoldij_temp)
233         MBoldij_list_x_y_z.append(np.array([MBold12, -0.5*(MBold11 -
234         MBold22), 0.5*(MBold11+MBold22)]))
235
236     return(MBoldij_list)
237
238 def prodscalm2D(M, N):
239     res = M[0,0]*N[0,0] + M[1,0]*N[0,1] + M[0,1]*N[1,0] + M[1,1]*N[1,1]
240     return(res)
241
242 def face(M1, M2, M3, p):
243     logdet1 = math.log(np.linalg.det(M1))
244     logdet2 = math.log(np.linalg.det(M2))

```

```

240     logdet3 = math.log(np.linalg.det(M3))
241     M1inv = sp.linalg.inv(M1)
242     M2inv = sp.linalg.inv(M2)
243     M3inv = sp.linalg.inv(M3)
244
245     truc=[prodscalm2D(M1,M1inv),prodscalm2D(M1,M2inv),prodscalm2D(M1,
246     M3inv),\
247     prodscalm2D(M2,M1inv),prodscalm2D(M2,M2inv),prodscalm2D(M2,
248     M3inv),\
249     prodscalm2D(M3,M1inv),prodscalm2D(M3,M2inv),prodscalm2D(M3,
250     M3inv)]
251
252     critere = [logdet1 - p + max(truc),logdet2 - p + max(truc),logdet2
253     - p + max(truc)]
254     face = min(critere)
255     return face
256
257 def getListMxyz(H1,H1T,H2,H2T,SigmaBoldinv,array_list,Q):
258     M11 = 0.5*(H1T@SigmaBoldinv@H1 + H1T@SigmaBoldinv@H1)
259     M12 = 0.5*(H1T@SigmaBoldinv@H2 + H2T@SigmaBoldinv@H1)
260     M21 = 0.5*(H2T@SigmaBoldinv@H1 + H1T@SigmaBoldinv@H2)
261     M22 = 0.5*(H2T@SigmaBoldinv@H2 + H2T@SigmaBoldinv@H2)
262
263     MBoldij_list = []
264
265     MBold11, MBold12, MBold21, MBold22 = 0., 0., 0., 0.
266     #print(np.squeeze(array_list[0]))
267     MBoldij_temp = np.zeros((2,2))
268
269     MBoldij_list_x_y_z = []
270
271     for i in range(len(array_list)):
272         MBold11 = np.transpose(np.squeeze(array_list[i]))@M11@np.
273         squeeze(array_list[i])
274         MBold12 = np.transpose(np.squeeze(array_list[i]))@M12@np.
275         squeeze(array_list[i])
276         MBold21 = np.transpose(np.squeeze(array_list[i]))@M21@np.
277         squeeze(array_list[i])
278         MBold22 = np.transpose(np.squeeze(array_list[i]))@M22@np.
279         squeeze(array_list[i])
280         MBoldij_temp[0,0] = MBold11 + Q[0,0]
281         MBoldij_temp[0,1] = MBold12 + Q[0,1]
282         MBoldij_temp[1,0] = MBold21 + Q[1,0]
283         MBoldij_temp[1,1] = MBold22 + Q[0,0]
284         MBoldij_list.append(MBoldij_temp)
285         MBoldij_list_x_y_z.append(np.array([-MBold12,-0.5*(MBold11-
286         MBold22),0.5*(MBold11+MBold22)]))
287
288     return(MBoldij_list_x_y_z)
289
290 def calculate_matrix(M):
291     """Given a point M = (x, y, z), compute the matrix Mi"""
292     x, y, z = M
293     return np.array([[z - y, x], [x, z + y]])
294
295 def log_det_maximum(M1, M2, M3, num_points=10):
296     # List to store log(det(M)) values and corresponding weights
297     log_dets = []

```

```

289
290     # Generate all possible weight combinations for w1, w2, w3 such
that w1 + w2 + w3 = 1
291     for i in range(num_points + 1):
292         w1 = i / num_points # w1 in the range [0, 1]
293         for j in range(num_points + 1 - i):
294             w2 = j / num_points # w2 in the range [0, 1]
295             w3 = 1 - w1 - w2 # w3 to make sure w1 + w2 + w3 = 1
296
297             if w3 >= 0: # Ensure all weights are non-negative
298                 # Calculate the weighted centroid point
299                 weighted_point = w1 * np.array(M1) + w2 * np.array(M2)
+ w3 * np.array(M3)
300
301                 # Calculate the corresponding matrix for the weighted
point
302                 M_matrix = calculate_matrix(weighted_point)
303
304                 # Compute the determinant of the matrix
305                 det = np.linalg.det(M_matrix)
306
307                 # If det is positive, compute its log (to avoid issues
with log(0) or negative det)
308                 if det > 0:
309                     log_det = math.log(det)
310                 else:
311                     log_det = -np.inf # If determinant is non-positive
, log(det) is not valid
312
313                 # Store the log(det) and corresponding weights
314                 log_dets.append((log_det, w1, w2, w3))
315
316     # Find the maximum log(det) and its corresponding weights
317     max_log_det = max(log_dets, key=lambda x: x[0])
318
319     return max_log_det
320
321 # def prodscalm2D(M,N):
322 #     res = M[0,0]*N[0,0] + M[1,0]*N[0,1] + M[0,1]*N[1,0] + M[1,1]*N
[1,1]
323 #     return(res)
324 def face(m1,m2,m3,p):
325     M1=calculate_matrix(m1)
326     M2=calculate_matrix(m2)
327     M3=calculate_matrix(m3)
328     logdet1 = math.log(np.linalg.det(M1))
329     logdet2 = math.log(np.linalg.det(M2))
330     logdet3 = math.log(np.linalg.det(M3))
331     M1inv = sp.linalg.inv(M1)
332     M2inv = sp.linalg.inv(M2)
333     M3inv = sp.linalg.inv(M3)
334
335     truc=[prodscalm2D(M1,M1inv),prodscalm2D(M1,M2inv),prodscalm2D(M1,
M3inv),\
336           prodscalm2D(M2,M1inv),prodscalm2D(M2,M2inv),prodscalm2D(M2,
M3inv),\
337           prodscalm2D(M3,M1inv),prodscalm2D(M3,M2inv),prodscalm2D(M3,
M3inv)]

```

```

338
339     critere = [logdet1 - p + max(truc), logdet2 - p + max(truc), logdet2
340 - p + max(truc)]
341     face = min(critere)
342     return face
343
344 #
345 #####
346
347 def getOptimalControl(p1,p2):
348     N=10
349     n=2
350     m=1
351     p=2
352     q=1
353     lsigmap=1e-4
354     lsigma=.25
355
356     t0=0.
357     tf=1.
358     dt=0.0001
359
360     Sigmap = np.array(lsigmap)
361     sigma = lsigma*np.eye(n)
362
363
364     A0=np.array([[ -1., 1.],[0., 1.]])
365     A1=np.eye((2))
366     A2=np.array([[0., -1.],[1., 0.]])
367     A=A0+p1*A1+p2*A2
368
369     F = sp.linalg.expm(tf*A)
370
371     B=np.array([[1.],[0.]])
372     C=np.array([[0., 1.]])
373
374     Bp = np.vstack([B,np.zeros_like(B)])
375     Cp = np.hstack([np.zeros_like(C),C])
376     newCp = np.hstack([C,np.zeros_like(C)])
377     newCpT=newCp.T
378
379     zeros = np.zeros_like(A)
380
381     Ap1 = np.vstack((np.hstack((A,zeros)),np.hstack((A1,A))))
382     Ap2 = np.vstack((np.hstack((A,zeros)),np.hstack((A2,A))))
383
384     Fp1 = sp.linalg.expm(tf*Ap1)
385     Fp2 = sp.linalg.expm(tf*Ap2)
386
387     L1 = Fp1[-n:,0:n]
388     L2 = Fp2[-n:,0:n]
389
390     # U(s)          1x1 , exemple simple U(s) = 1
391     U = lambda t: 1.
392

```

```

393 u=np.zeros(10)
394 u[0],u[1]=0.70710678,0.70710678
395 sigma_sigma_T = sigma@np.transpose(sigma)
396
397 #
#####

398
399 def f1(G,t):
400     return Ap1 @ G + np.dot(Bp , U(t))
401
402 def f1(G,t):
403     return Ap1 @ G + np.dot(Bp , U(t))
404 def f2(G,t):
405     return Ap2 @ G + np.dot(Bp , U(t))
406
407 Gp1 = getGi(f1,n,m,t0,tf,dt)
408 Gp2 = getGi(f2,n,m,t0,tf,dt)
409 #
#####

410
411
412 def f(S):
413     return A @ S + S @ A.T + sigma_sigma_T
414
415 Sigma = getSigma(f,n,t0,tf,dt)
416
417 #####
418
419 Array=np.hstack((np.vstack((sigma_sigma_T,np.zeros_like(
sigma_sigma_T))),np.vstack((np.zeros_like(sigma_sigma_T),np.
zeros_like(sigma_sigma_T)))))
420 #print(Array)
421
422 def fs1(S):
423     return Ap1 @ S + S @ (Ap1.T) + Array
424
425 Sigma1=getSi(fs1,n,t0,tf,dt)
426 #OK
427
428 def fs2(S):
429     return Ap2 @ S + S @ (Ap2.T) + Array
430 Sigma2=getSi(fs2,n,t0,tf,dt)
431 #####
432
433
434 SigmaBold = getSigmaBold(Sigma,N,F,C,lsigma)
435
436 SigmaBoldinv=np.linalg.inv(SigmaBold)
437
438 #####
439 H1,H2 = getH1H2(N,Fp1,Fp2,Gp1,Gp2,Cp)
440
441 #####
442
443 Sigma1Bold = getSigmaiBold(C,Cp,newCp,newCpT,Sigma1,n,N,Fp1)
444 Sigma2Bold = getSigmaiBold(C,Cp,newCp,newCpT,Sigma2,n,N,Fp2)

```

```

445 #####
446 H1T=H1.T
447 H2T=H2.T
448 Q=getQ(SigmaBoldinv,Sigma1Bold,Sigma2Bold)
449
450 # G n re toutes les combinaisons possibles de 0 et 1 sur 10
451 positions
452 combinations = list(itertools.product([0., 1.], repeat=10))
453
454 # Cr e une liste de tableaux numpy (1x10) pour chaque combinaison
455 pre_array_list = [np.array([combo]) for combo in combinations]
456
457 # Step 3: Normalize each array by the number of 1s (L2
458 normalization)
459 array_list = [ arr for arr in pre_array_list]
460
461 ListM=getListM(H1,H1T,H2,H2T,SigmaBoldinv,array_list,Q)
462
463 #####
464
465 def fg(G,t):
466     return A @ G + np.dot(B , U(t))
467
468 G= getG(fg,n,m,t0,tf,dt)
469 #####
470 listMplott=getListMxyz(H1,H1T,H2,H2T,SigmaBoldinv,array_list,Q)
471 listMplot=np.array(listMplott)
472
473 # Compute the convex hull
474 hull = ConvexHull(listMplot)
475
476 #
477 #####
478
479 list_logdet_full=[]
480 list_logdet=[]
481
482 #print(len(hull.vertices))
483 for k in range(len(hull.vertices)):
484     temp=calculate_matrix(listMplot[hull.vertices[k]])
485     det = np.linalg.det(temp)
486     # If det is positive, compute its log (to avoid
487     issues with log(0) or negative det)
488     if det > 0:
489         log_det = math.log(det)
490     else:
491         log_det = -np.inf
492     list_logdet_full.append([k,temp,det,log_det])
493     list_logdet.append(log_det)
494
495 maxlogdet_verticies=max(list_logdet)
496
497

```



```

498
499     hullnew_simplices = []
500     for i in range(len(hull.simplices)):
501         if(hull.simplices[i][0] != 0 and hull.simplices[i][1] != 0 and
hull.simplices[i][2] != 0 and hull.simplices[i][0] != 1 and hull.
simplices[i][1] != 1 and hull.simplices[i][2] != 1):
502             hullnew_simplices.append([hull.simplices[i][0],hull.
simplices[i][1],hull.simplices[i][2]])
503
504
505     indiceListDesPointsGardes=[]
506     listDesPointsGardes=[]
507
508     #boucle sur les faces
509     for k in range(len(hullnew_simplices)): #deux premier det < 0
510         facetemp = face(listMplot[hullnew_simplices[k][0]],listMplot[
hullnew_simplices[k][1]],listMplot[hullnew_simplices[k][2]],p)
511         if(facetemp > maxlogdet_verticies):
512             indiceListDesPointsGardes.append([k,facetemp])
513             listDesPointsGardes.append([listMplot[hullnew_simplices[k]
][0]],listMplot[hullnew_simplices[k][1]],listMplot[hullnew_simplices
[k][2]])]
514
515
516     #listDesPointsGardes x,y,z avec matrice [[z+y , x],[x , z-y]]
517     listwi0ptSurFace=[]
518     listmax=[]
519     for k in range(len(listDesPointsGardes)):
520         M1,M2,M3=listDesPointsGardes[k]
521         # Get the maximum log determinant and the corresponding weights
522         max_log_det, w1, w2, w3 = log_det_maximum(M1, M2, M3,
num_points=40)
523         listwi0ptSurFace.append([k,max_log_det, w1, w2, w3,M1,M2,M3])
524         listmax.append(max_log_det)
525
526     #print(listmax,max(listmax))
527     un,deux,trois=hullnew_simplices[listwi0ptSurFace[max_log_det==max(
listmax)][0]]
528
529     k,max_log_det, w1, w2, w3,M1,M2,M3=listwi0ptSurFace[max_log_det==
max(listmax)]
530
531     #print('control',w1*array_list[un] + w2*array_list[deux] + w3*
array_list[trois])
532
533     u1,u2,u3 = np.squeeze(array_list[un]) ,np.squeeze(array_list[deux])
,np.squeeze(array_list[trois])
534
535     return u1,u2,u3,w1,w2,w3,SigmaBold,SigmaBoldinv,Sigma1Bold,
Sigma2Bold
536
537 def gety(truep1,truep2,control):
538     N=10
539     n=2
540     m=1
541     p=2
542     q=1
543     lsigmap=1e-4

```

```

544     lsigma=.25
545
546     p1,p2=truep1,truep2
547
548     t0=0.
549     tf=1.
550     dt=0.0001
551
552
553     Sigmap = np.array(lsigmap)
554     sigma = lsigma*np.eye(n)
555
556
557     A0=np.array([[ -1.,1.],[0.,1.]])
558     A1=np.eye((2))
559     A2=np.array([[0.,-1.],[1.,0.]])
560     A=A0+p1*A1+p2*A2
561
562     F = sp.linalg.expm(tf*A)
563
564     B=np.array([[1.],[0.]])
565     C=np.array([[0.,1.]])
566
567     Bp = np.vstack([B,np.zeros_like(B)])
568     Cp = np.hstack([np.zeros_like(C),C])
569     newCp = np.hstack([C,np.zeros_like(C)])
570     newCpT=newCp.T
571
572     zeros = np.zeros_like(A)
573
574     Ap1 = np.vstack((np.hstack((A,zeros)),np.hstack((A1,A))))
575     Ap2 = np.vstack((np.hstack((A,zeros)),np.hstack((A2,A))))
576
577     Fp1 = sp.linalg.expm(tf*Ap1)
578     Fp2 = sp.linalg.expm(tf*Ap2)
579
580     L1 = Fp1[-n:,0:n]
581     L2 = Fp2[-n:,0:n]
582
583     # U(s)          1x1 , exemple simple U(s) = 1
584     U = lambda t: 1.
585
586     sigma_sigma_T = sigma@np.transpose(sigma)
587
588     #
589     #####
590
591     def f1(G,t):
592         return Ap1 @ G + np.dot(Bp , U(t))
593
594     def f1(G,t):
595         return Ap1 @ G + np.dot(Bp , U(t))
596     def f2(G,t):
597         return Ap2 @ G + np.dot(Bp , U(t))
598
599     Gp1 = getGi(f1,n,m,t0,tf,dt)
600     Gp2 = getGi(f2,n,m,t0,tf,dt)

```

```

600
601     def fg(G,t):
602         return A @ G + np.dot(B , U(t))
603
604     G = getG(fg,n,m,t0,tf,dt)
605     #
606     #####
607
608     def f(S):
609         return A @ S + S @ A.T + sigma_sigma_T
610
611     Sigma = getSigma(f,n,t0,tf,dt)
612
613     x0=np.zeros((n,1))
614
615     x=np.zeros((n,10))
616     x[0,0],x[1,0]=x0[0],x0[1]
617
618     start=0.
619     end=10.
620     delta = 1.
621     time=np.arange(start,end,delta)
622     np.random.seed(seed=1)
623
624     Cov = np.eye(2)
625     Cov[0,0] = np.sqrt(Sigma[0,0])
626     Cov[1,1] = np.sqrt(Sigma[1,1])
627
628     for k in range(10-1):
629         dW= np.random.multivariate_normal(mean = np.zeros(2) , cov=Cov)
630         Fxk=np.dot(F,x[:,k].reshape(2,1))
631         x[0,k+1],x[1,k+1] = Fxk + np.dot(G.reshape(2,1),control[k]) +
632         dW.reshape(2,1)
633         return(time,x[1,:])
634     #####
635     def getetha(p1,p2,control):
636         N=10
637         n=2
638         m=1
639         p=2
640         q=1
641         lsigmap=1e-4
642         lsigma=.25
643
644         t0=0.
645         tf=1.
646         dt=0.0001
647
648
649         Sigmap = np.array(lsigmap)
650         sigma = lsigma*np.eye(n)
651
652
653         A0=np.array([[ -1.,1.],[0.,1.]])
654         A1=np.eye((2))

```

```

655 A2=np.array([[0.,-1.],[1.,0.]])
656 A=A0+p1*A1+p2*A2
657
658 F = sp.linalg.expm(tf*A)
659
660 B=np.array([[1.],[0.]])
661 C=np.array([[0.,1.]])
662
663 Bp = np.vstack([B,np.zeros_like(B)])
664 Cp = np.hstack([np.zeros_like(C),C])
665 newCp = np.hstack([C,np.zeros_like(C)])
666 newCpT=newCp.T
667
668 zeros = np.zeros_like(A)
669
670 Ap1 = np.vstack((np.hstack((A,zeros)),np.hstack((A1,A))))
671 Ap2 = np.vstack((np.hstack((A,zeros)),np.hstack((A2,A))))
672
673 Fp1 = sp.linalg.expm(tf*Ap1)
674 Fp2 = sp.linalg.expm(tf*Ap2)
675
676 L1 = Fp1[-n:,0:n]
677 L2 = Fp2[-n:,0:n]
678
679 # U(s)          1x1 , exemple simple U(s) = 1
680 U = lambda t: 1.
681
682 u=np.zeros(10)
683 u[0],u[1]=0.70710678,0.70710678
684 sigma_sigma_T = sigma@np.transpose(sigma)
685
686 #
687 #####
688
689 def f1(G,t):
690     return Ap1 @ G + np.dot(Bp , U(t))
691
692 def f1(G,t):
693     return Ap1 @ G + np.dot(Bp , U(t))
694
695 def f2(G,t):
696     return Ap2 @ G + np.dot(Bp , U(t))
697
698 Gp1 = getGi(f1,n,m,t0,tf,dt)
699 Gp2 = getGi(f2,n,m,t0,tf,dt)
700 #
701 #####
702
703 def f(S):
704     return A @ S + S @ A.T + sigma_sigma_T
705
706 Sigma = getSigma(f,n,t0,tf,dt)
707
708 #####
709
710 Array=np.hstack((np.vstack((sigma_sigma_T,np.zeros_like(

```

```

sigma_sigma_T))) , np.vstack((np.zeros_like(sigma_sigma_T), np.
zeros_like(sigma_sigma_T))))
709 #print(Array)
710
711 def fs1(S):
712     return Ap1 @ S + S @ (Ap1.T) + Array
713
714 Sigma1=getSi(fs1,n,t0,tf,dt)
715 #OK
716
717 def fs2(S):
718     return Ap2 @ S + S @ (Ap2.T) + Array
719 Sigma2=getSi(fs2,n,t0,tf,dt)
720 #####
721
722 SigmaBold = getSigmaBold(Sigma,N,F,C,lsigmap)
723
724 SigmaBoldinv=np.linalg.inv(SigmaBold)
725
726 #####
727 H1,H2 = getH1H2(N,Fp1,Fp2,Gp1,Gp2,Cp)
728
729
730 def fg(G,t):
731     return A @ G + np.dot(B , U(t))
732
733 G = getG(fg,n,m,t0,tf,dt)
734
735
736 etha=[]
737 etha.append(np.squeeze(np.zeros(1)))
738 for k in range(1,10):
739     sum=0
740     for l in range(k):
741         sum = sum + C@(np.linalg.matrix_power(F,k-l-1))@(np.dot(G,
control[1]))
742     etha.append(np.squeeze(sum))
743
744 dethap1 = H1@control
745 dethap2 = H2@control
746 return np.array(etha), np.array(dethap1), np.array(dethap2)
747
748
749 #####
750 def getJ(y,etha,SigmaBold,SigmaBoldinv):
751     logdetSigma=math.log(np.linalg.det(SigmaBold))
752     yetha= y-etha
753     return (yetha.T)@SigmaBoldinv@yetha + logdetSigma
754
755
756 #####
757 def getJp1Jp2(y,etha,dethap1,dethap2,SigmaBoldinv,Sigma1Bold,Sigma2Bold
):
758     yetha= y-etha
759     yethaT = yetha.T
760
761     A1 = -2. * (yethaT@(SigmaBoldinv@dethap1))
762     A2 = -2. * (yethaT@(SigmaBoldinv@dethap2))

```

```

763
764     B1 = yethaT@(SigmaBoldinv@Sigma1Bold@SigmaBoldinv)@yetha
765     B2 = yethaT@(SigmaBoldinv@Sigma2Bold@SigmaBoldinv)@yetha
766
767     C1=np.trace(SigmaBoldinv@Sigma1Bold)
768     C2=np.trace(SigmaBoldinv@Sigma2Bold)
769
770     Jp1 = A1 + B1 + C1
771     Jp2 = A2 + B2 + C2
772
773     return Jp1,Jp2

```

Voici le code Python de la deuxième partie:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import scipy as sp
4  import itertools
5  import math
6  from tools import *
7  from scipy.spatial import ConvexHull
8  from mpl_toolkits.mplot3d import Axes3D
9  from matplotlib import cm
10 from matplotlib.colors import LightSource
11
12
13 N=10
14 n=2
15 m=1
16 p=2
17 q=1
18 lsigmap=1e-4
19 lsigma=.01
20
21 t0=0.
22 tf=1.
23 dt=0.0001
24
25 p1=-.1
26 p2= 2.
27
28
29 Sigmap = np.array(lsigmap)
30 sigma = lsigma*np.eye(n)
31
32
33 A0=np.array([[ -1.,1.],[0.,1.]])
34 A1=np.eye((2))
35 A2=np.array([[0.,-1.],[1.,0.]])
36 A=A0+p1*A1+p2*A2
37
38 F = sp.linalg.expm(tf*A)
39
40 B=np.array([[1.],[0.]])
41 C=np.array([[0.,1.]])
42
43 Bp = np.vstack([B,np.zeros_like(B)])
44 Cp = np.hstack([np.zeros_like(C),C])
45 newCp = np.hstack([C,np.zeros_like(C)])

```

```

46 newCpT=newCp.T
47
48 zeros = np.zeros_like(A)
49
50 Ap1 = np.vstack((np.hstack((A,zeros)),np.hstack((A1,A))))
51 Ap2 = np.vstack((np.hstack((A,zeros)),np.hstack((A2,A))))
52
53 Fp1 = sp.linalg.expm(tf*Ap1)
54 Fp2 = sp.linalg.expm(tf*Ap2)
55
56 L1 = Fp1[-n:,0:n]
57 L2 = Fp2[-n:,0:n]
58
59 # U(s)          1x1 , exemple simple U(s) = 1
60 U = lambda t: 1.
61
62 u=np.zeros(10)
63 u[0],u[1]=0.70710678,0.70710678
64 sigma_sigma_T = sigma@np.transpose(sigma)
65
66 #
67 #####
68
69 def f1(G,t):
70     return Ap1 @ G + np.dot(Bp , U(t))
71
72 def f1(G,t):
73     return Ap1 @ G + np.dot(Bp , U(t))
74
75 def f2(G,t):
76     return Ap2 @ G + np.dot(Bp , U(t))
77
78 Gp1 = getGi(f1,n,m,t0,tf,dt)
79 Gp2 = getGi(f2,n,m,t0,tf,dt)
80
81 #
82 #####
83
84 def f(S):
85     return A @ S + S @ A.T + sigma_sigma_T
86
87 Sigma = getSigma(f,n,t0,tf,dt)
88
89 #####
90
91 Array=np.hstack((np.vstack((sigma_sigma_T,np.zeros_like(sigma_sigma_T))
92     ),np.vstack((np.zeros_like(sigma_sigma_T),np.zeros_like(
93     sigma_sigma_T))))))
94 #print(Array)
95
96 def fs1(S):
97     return Ap1 @ S + S @ (Ap1.T) + Array
98
99 Sigma1=getSi(fs1,n,t0,tf,dt)
100 #OK
101
102 def fs2(S):

```

```

98     return Ap2 @ S + S @ (Ap2.T) + Array
99 Sigma2=getSi(fs2,n,t0,tf,dt)
100 #####
101
102
103 SigmaBold = getSigmaBold(Sigma,N,F,C,lsigma)
104
105 SigmaBoldinv=np.linalg.inv(SigmaBold)
106
107 #####
108 H1,H2 = getH1H2(N,Fp1,Fp2,Gp1,Gp2,Cp)
109
110 #####
111
112 Sigma1Bold = getSigmaBold(C,Cp,newCp,newCpT,Sigma1,n,N,Fp1)
113 Sigma2Bold = getSigmaBold(C,Cp,newCp,newCpT,Sigma2,n,N,Fp2)
114
115 #####
116 H1T=H1.T
117 H2T=H2.T
118 Q=getQ(SigmaBoldinv,Sigma1Bold,Sigma2Bold)
119
120 # G n re toutes les combinaisons possibles de 0 et 1 sur 10 positions
121 combinations = list(itertools.product([0., 1.], repeat=10))
122
123 # Cr e une liste de tableaux numpy (1x10) pour chaque combinaison
124 pre_array_list = [np.array([combo]) for combo in combinations]
125
126
127 # Step 3: Normalize each array by the number of 1s (L2 normalization)
128 array_list = [
129     arr / np.sqrt(np.sum(arr**2)) if np.sum(arr**2) != 0 else arr #
130     Avoid division by zero
131     for arr in pre_array_list
132 ]
133 ListM=getListM(H1,H1T,H2,H2T,SigmaBoldinv,array_list,Q)
134
135
136 #####
137
138 def fg(G,t):
139     return A @ G + np.dot(B , U(t))
140
141 G= getG(fg,n,m,t0,tf,dt)
142 #####
143 listMplott=getListMxyz(H1,H1T,H2,H2T,SigmaBoldinv,array_list,Q)
144 listMplot=np.array(listMplott)
145
146 fig = plt.figure()
147 ax = fig.add_subplot(111, projection='3d')
148
149 # Create a 2D grid
150 XX = np.linspace(-30000, 60000, 100)
151 YY = np.linspace(-25000, 100000, 100)
152 X, Y = np.meshgrid(XX, YY)
153
154 # Compute the distance from the origin

```



```

155 Z = np.sqrt(X**2 + Y**2)
156
157
158 # Light source for realistic shading
159 ls = LightSource(azdeg=0, altdeg=65)
160 rgb = ls.shade(Z, cmap=cm.coolwarm, vert_exag=0.1, blend_mode='soft')
161
162 # Plot cone surface
163 ax.plot_surface(X, Y, Z, rstride=1, cstride=1, facecolors=rgb, alpha
    =0.5, antialiased=True, linewidth=0)
164
165
166 # Scatter plot des points
167 ax.scatter(listMplot[:, 0], listMplot[:, 1], listMplot[:, 2], s=12, c=
    'black', alpha=0.4, depthshade=True)
168
169 # Compute the convex hull
170 hull = ConvexHull(listMplot)
171
172 for simplex in hull.simplices:
173     tri = listMplot[simplex]
174     ax.plot_trisurf(tri[:, 0], tri[:, 1], tri[:, 2], color='maroon',
        alpha=0.8, edgecolor='none', linewidth=0, zorder=10)
175
176
177 # Aesthetic axis labels and limits
178 ax.set_xlabel('X', fontsize=12, labelpad=10)
179 ax.set_ylabel('Y', fontsize=12, labelpad=10)
180 ax.set_zlabel('Z', fontsize=12, labelpad=10)
181
182 # Optional: Adjust viewing angle
183 ax.view_init(elev=40, azimuth=45)
184
185 plt.show()
186
187 #
188 #####
189
190 list_logdet_full=[]
191 list_logdet=[]
192
193 #print(len(hull.vertices))
194 for k in range(len(hull.vertices)):
195     temp=calculate_matrix(listMplot[hull.vertices[k]])
196     det = np.linalg.det(temp)
197     # If det is positive, compute its log (to avoid
198     # issues with log(0) or negative det)
199     if det > 0:
200         log_det = math.log(det)
201     else:
202         log_det = -np.inf
203     list_logdet_full.append([k,temp,det,log_det])
204     list_logdet.append(log_det)
205
206 maxlogdet_verticies=max(list_logdet)

```

```

207 hullnew_simplices = []
208 for i in range(len(hull.simplices)):
209     if(hull.simplices[i][0] != 0 and hull.simplices[i][1] != 0 and hull.
        simplices[i][2] != 0 and hull.simplices[i][0] != 1 and hull.simplices[
        i][1] != 1 and hull.simplices[i][2] != 1):
210         hullnew_simplices.append([hull.simplices[i][0],hull.simplices[i
        ][1],hull.simplices[i][2]])
211
212
213 indiceListDesPointsGardes=[]
214 listDesPointsGardes=[]
215
216 #boucle sur les faces
217 for k in range(len(hullnew_simplices)): #deux premier det < 0
218     facetemp = face(listMplot[hullnew_simplices[k][0]],listMplot[
        hullnew_simplices[k][1]],listMplot[hullnew_simplices[k][2]],p)
219     if(facetemp > maxlogdet_verticies):
220         indiceListDesPointsGardes.append([k,facetemp])
221         listDesPointsGardes.append([listMplot[hullnew_simplices[k][0]],
        listMplot[hullnew_simplices[k][1]],listMplot[hullnew_simplices[k
        ][2]]])
222
223
224 #listDesPointsGardes x,y,z avec matrice [[z+y , x],[x , z-y]]
225 listwi0ptSurFace=[]
226 listmax=[]
227 for k in range(len(listDesPointsGardes)):
228     M1,M2,M3=listDesPointsGardes[k]
229     # Get the maximum log determinant and the corresponding weights
230     max_log_det, w1, w2, w3 = log_det_maximum(M1, M2, M3, num_points
        =40)
231     listwi0ptSurFace.append([k,max_log_det, w1, w2, w3,M1,M2,M3])
232     listmax.append(max_log_det)
233
234 #print(listmax,max(listmax))
235 un,deux,trois=hullnew_simplices[listwi0ptSurFace[max_log_det==max(
        listmax)][0]]
236
237 k,max_log_det, w1, w2, w3,M1,M2,M3=listwi0ptSurFace[max_log_det==max(
        listmax)]
238
239 print('control',w1*array_list[un] + w2*array_list[deux] + w3*array_list
        [trois])
240
241 control = (w1*array_list[un] + w2*array_list[deux] + w3*array_list[
        trois])[0]
242
243
244 # Create a 3D plot
245 fig = plt.figure(22)
246 ax = fig.add_subplot(111, projection='3d')
247
248 # Plot the listMplot
249 ax.scatter(listMplot[:, 0], listMplot[:, 1], listMplot[:, 2], color='
        black')
250 ax.scatter(listMplot[768, 0], listMplot[768, 1], listMplot[768, 2],
        color='red')
251 # Plot the convex hull faces with colors based on their position

```

```

relative to the median
252 for simplex in hull.simplices:
253     ax.plot_trisurf(listMplot[simplex, 0], listMplot[simplex, 1],
        listMplot[simplex, 2], color='maroon', alpha=0.5)
254 plt.show()
255
256
257 x0=np.zeros((n,1))
258
259 x=np.zeros((n,10))
260 x[0,0],x[1,0]=x0[0],x0[1]
261
262 start=0.
263 end=10.
264 delta = 1.
265 time=np.arange(start,end,delta)
266 np.random.seed(seed=1)
267
268 dW= np.array(np.random.multivariate_normal(mean = np.zeros(2) , cov=
        Sigma))
269
270 for k in range(10-1):
271     dW= np.random.multivariate_normal(mean = np.zeros(2) , cov=Sigma)
272     Fxk=np.dot(F,x[:,k].reshape(2,1))
273     x[0,k+1],x[1,k+1] = Fxk + np.dot(G.reshape(2,1),control[k]) + dW.
        reshape(2,1)
274
275 plt.figure(50)
276 plt.plot(time,x[0,:])
277 plt.plot(time,x[1,:])
278 plt.show()

```